TOPIC DRILL · ทบทวนทีละเนื้อหา

โจทย์แยกตามเนื้อหา พร้อมสรุปและเฉลย

รวมโจทย์จาก Mock Exam ชุดที่ 1–4 และโจทย์เพิ่มเติม จัดเป็น 15 หัวข้อ แต่ละหัวข้อ: สรุปก่อนทำ → โจทย์ → เฉลยท้ายหัวข้อ

Digital Logic	Number systems · Boolean/K-map · Combinational · Sequential/FSM
Computer Architecture	Intro · Performance · ISA (2 ส่วน) · Single cycle · Multi cycle · Pipelining · ILP · Cache · Virtual memory · Storage
เฉลย	อยู่ท้ายแต่ละหัวข้อ (เริ่มหน้าใหม่) มีตารางคำตอบและเหตุผลรายข้อ

ข้อตกลง: byte addressing · 1 word = 4 bytes · 1 KB = 2^{10} B · nanoLADA ตามสไลด์ (LW 5 / ORI, ADD, SW 4 / BEQ, JMP 3 cycles)

สารบัญ

วิธีใช้ Topic Drill

DL1: Number Systems และ Signed Numbers

DL2: Boolean Algebra และ K-map

DL3: Combinational Circuits

DL4: Sequential Circuits และ FSM

Ch1: Introduction (ISA vs Organization)

Ch2: Performance (Speedup, CPU Time, Amdahl)

Ch3 (1): ISA Types, CISC/RISC, Format และ Compiler

Ch3 (2): Endian, Alignment, Addressing, Control Flow, Exceptions

Ch4: Single Cycle Processor (nanoLADA)

Ch5: Multiple Cycle Processor

Ch6: Pipelining และ Hazards

Ch7: Instruction-Level Parallelism

Ch8 (1): Memory Hierarchy และ Cache

Ch8 (2): Virtual Memory และ TLB

Ch9: Storage (RAID, SAN, NAS)

วิธีใช้ Topic Drill

เล่มนี้รวมโจทย์จาก Mock Exam ชุดที่ 1–4 (ดัดแปลงบางข้อ) และเติมโจทย์ใหม่ในจุดที่ mock ยังไม่ครอบคลุม แล้ว**จัดกลุ่มตามเนื้อหา** เพื่อให้ทบทวนทีละเรื่องได้จนแน่น

โครงสร้างของแต่ละหัวข้อ

1. **สรุปก่อนทำโจทย์:** สูตรและข้อเท็จจริงที่ต้องใช้ในหัวข้อนั้น (อ่านคู่กับบทเดียวกันใน Exam Notes)
2. **โจทย์:** เรียงจากแนวคิด → คำนวณ → ตอบสั้น แบ่งเป็นกลุ่มย่อย ก, ข, ค, ...
3. **เฉลย:** เริ่มหน้าใหม่ท้ายหัวข้อ มีตารางคำตอบ Choice สำหรับตรวจเร็ว แล้วตามด้วยเหตุผลรายข้อ (บอกตัวลวงด้วย)

เลขข้อมีตัวอักษรนำหน้าตามหัวข้อ เช่น **P5** = Performance ข้อ 5 · **K12** = Cache ข้อ 12

รายการหัวข้อ

ส่วน	หัวข้อ	รหัส
Digital Logic	DL1 Number Systems และ Signed Numbers	N
	DL2 Boolean Algebra และ K-map	B
	DL3 Combinational Circuits	C
	DL4 Sequential Circuits และ FSM	F
Computer Architecture	Ch1 Introduction	I
	Ch2 Performance	P
	Ch3 (1) ISA Types, CISC/RISC, Format, Compiler	A
	Ch3 (2) Endian, Alignment, Addressing, Exceptions	D
	Ch4 Single Cycle (nanoLADA)	S
	Ch5 Multiple Cycle	M
	Ch6 Pipelining และ Hazards	L
	Ch7 ILP	X
	Ch8 (1) Cache	K
	Ch8 (2) Virtual Memory และ TLB	V
	Ch9 Storage	R

แนะนำวิธีทบทวน

- **ทำทีละหัวข้อ:** อ่านสรุป → ทำโจทย์ทั้งหมดโดยไม่ดูเฉลย → ตรวจ → ข้อที่ผิดให้กลับไปอ่านบทเดียวกันใน Exam Notes แล้วทำข้อนั้นใหม่

- **ลำดับที่แนะนำถ้าเวลาน้อย:** P → L → K → S/M → D → V → X → R → A → I แล้วจึง Digital Logic (หัวข้อแรก ๆ เป็นโจทย์คำนวณที่ออกบ่อยในข้อตอบสั้น)
- **หลังทบทวนครบทุกหัวข้อ:** ทำ Mock Exam ชุดใดชุดหนึ่งแบบจับเวลา 90 นาที เพื่อฝึกการคุมเวลาจริง
- โจทย์ในเล่มนี้แต่งขึ้นตามแนวของสไลด์และ Exercise ท้ายบท ไม่ใช่ข้อสอบเก่า

DL1: Number Systems และ Signed Numbers

สรุปก่อนทำโจทย์

- **แปลงฐาน:** ฐานใด ๆ → ฐานสิบให้คูณด้วยค่าประจำหลัก (หลังจุดคือ 2^{-1}, 2^{-2}, ...) ส่วนฐานสิบ → ฐานสองให้หาร 2 แล้วอ่านเศษจาก**ล่างขึ้นบน** และสำหรับทศนิยมให้คูณ 2 แล้วอ่านหลักหน้าจุดจาก**บนลงล่าง**
- **ฐาน 16 จัดกลุ่มละ 4 บิต ฐาน 8 จัดกลุ่มละ 3 บิต** เริ่มจัดจากจุดทศนิยม
- **ช่วงค่า n bits:** unsigned $0 \ldots 2^n - 1$ · sign-magnitude และ 1's complement $-(2^{n-1} - 1) \ldots 2^{n-1} - 1$ · **2's complement $-2^{n-1} \ldots 2^{n-1} - 1$** (ไม่สมมาตร มีศูนย์ตัวเดียว)
- **2's complement ของ −X:** กลับทุกบิตแล้วบวก 1 ส่วน **1's complement:** กลับทุกบิตอย่างเดียว
- **Sign extension:** คัดลอก sign bit ไปทางซ้าย ค่าไม่เปลี่ยน (ใช้กับ `sign_ext` ของ nanoLADA)
- **Carry out** มีความหมายกับ unsigned ส่วน **overflow (signed)** เกิดเมื่อ บวก + บวก ได้ลบ หรือ ลบ + ลบ ได้บวก (= $C_{in,MSB} \oplus C_{out,MSB}$) และ**บวกเลขต่างเครื่องหมายไม่มีทาง overflow**
- **BCD:** 4 บิตต่อเลขฐานสิบ 1 หลัก · **Gray code:** ค่าติดกันต่างกัน 1 บิต $g = b \oplus (b >> 1)$

โจทย์ (18 ข้อ)

ก. แปลงฐาน

N1. $(1011.01)_2$ มีค่าเท่าใดในฐานสิบ

a) 11.01
b) 11.25
c) 11.5
d) 13.25

N2. $(2B)_{16}$ มีค่าเท่าใดในฐานสิบ

a) 32
b) 43
c) 53
d) 211

N3. $(1101\ 0110)_2$ เขียนเป็นฐานสิบหกได้ว่า

a) 0xD6
b) 0x6D
c) 0xC6
d) 0xD5

N4. 0x3F7 มีค่าเท่าใดในฐานแปด

a) 1767_8
b) 3767_8
c) 1773_8
d) 7731_8

N5. 345_8 มีค่าเท่าใดในฐานสิบ

a) 345
b) 229
c) 237
d) 197

N6. 25.625 ในฐานสิบ เขียนเป็นฐานสองได้ว่า

a) 11001.101
b) 11001.011
c) 10011.101
d) 11010.101

ข. ช่วงค่าและการแทนเลขลบ

N7. ช่วงค่าของเลข 6-bit two's complement คือ

a) –31 ถึง 31
b) –32 ถึง 31
c) –31 ถึง 32
d) 0 ถึง 63

N8. เลข unsigned 10 bits มีช่วงค่าเท่าใด

a) 0 ถึง 1023
b) –512 ถึง 511
c) 0 ถึง 1024
d) 0 ถึง 511

N9. –100 ในระบบ 8-bit two's complement คือ

a) 0x64
b) 0xE4
c) 0x9B
d) 0x9C

N10. 1111 0110 ในระบบ 8-bit two's complement มีค่าเท่าใด

a) –9
b) 246
c) –10
d) –118

N11. –90 ในระบบ 8-bit **one's complement** คือ

a) 1010 0110
b) 1010 0101
c) 1101 1010
d) 0101 1010

N12. Sign-extend เลข 4-bit two's complement `1010` ให้เป็น 8 bits ได้ผลเป็น

a) 0000 1010
b) 1000 1010
c) 1111 0101
d) 1111 1010

ค. Carry และ Overflow

N13. บวกเลข 8-bit two's complement 0x70 + 0x50 ข้อใดถูก

a) carry out = 1, overflow = 1
b) carry out = 0, overflow = 0
c) carry out = 0, overflow = 1
d) carry out = 1, overflow = 0

N14. บวกเลข 8-bit two's complement 0x90 + 0xA0 ข้อใดถูก

a) carry out = 0, overflow = 0
b) carry out = 1, overflow = 0
c) carry out = 0, overflow = 1
d) carry out = 1, overflow = 1

N15. การบวกเลข 8-bit two's complement คู่ใดเกิด signed overflow

a) 0x7F + 0x01

b) 0x80 + 0x7F

c) 0xFF + 0x01

d) 0x40 + 0xC0

ง. ตอบสั้น

N16. เขียน –37 ในระบบ 8-bit two's complement ทั้งฐานสองและฐานสิบหก

N17. (ก) แปลง binary 1011 เป็น Gray code (ข) เขียน 59 ในรูป BCD

N18. ทำไม two's complement จึงนิยมกว่า sign-magnitude และ 1's complement (ตอบ 2 เหตุผล)

เฉลย: DL1: Number Systems และ Signed Numbers

ข้อ	N1	N2	N3	N4	N5	N6	N7	N8	N9	N10
ตอบ	b	b	a	a	b	a	b	a	d	c

ข้อ	N11	N12	N13	N14	N15
ตอบ	b	d	c	d	a

N1. ตอบ b) 11.25

8 + 2 + 1 + 0.25 = 11.25 (หลังจุด $0.01_2 = 2^{-2} = 0.25$) ตัวลวง c คิดตำแหน่งหลังจุดผิด

N2. ตอบ b) 43

2 × 16 + 11 = 43

N3. ตอบ a) 0xD6

1101 = D และ 0110 = 6 (ตัวลวง b สลับลำดับ nibble)

N4. ตอบ a) 1767_8

0011 1111 0111 → จัดใหม่ทีละ 3 บิตเป็น 001 111 110 111 = 1767_8 (ตรวจ: ทั้งสองค่าเท่ากับ 1015)

N5. ตอบ b) 229

3 × 64 + 4 × 8 + 5 = 192 + 32 + 5 = 229

N6. ตอบ a) 11001.101

25 = 11001 และ 0.625 → ×2 = 1.25 (1), 0.25 → ×2 = 0.5 (0), 0.5 → ×2 = 1.0 (1) อ่านจากบนลงล่างได้ .101 (ตัวลวง b อ่านกลับด้าน)

N7. ตอบ b) –32 ถึง 31

-2^5 ถึง $2^5 - 1$ (ตัวลวง a เป็นช่วงของ sign-magnitude/1's complement และตัวลวง d เป็น unsigned)

N8. ตอบ a) 0 ถึง 1023

0 ถึง $2^{10} - 1$ (ตัวลวง c ลืมลบ 1)

N9. ตอบ d) 0x9C

+100 = 0110 0100 → กลับบิตได้ 1001 1011 (0x9B = 1's complement) → +1 ได้ 1001 1100 = 0x9C (ตรวจ: 156 − 256 = −100)

N10. ตอบ c) −10

กลับบิตแล้วบวก 1: 0000 1001 + 1 = 10 จึงได้ −10 (a = อ่านแบบ 1's complement, b = unsigned, d = sign-magnitude)

N11. ตอบ b) 1010 0101

+90 = 0101 1010 แล้วกลับทุกบิต (ตัวลวง a เป็น 2's complement)

N12. ตอบ d) 1111 1010

คัดลอก sign bit (1) ไปทางซ้าย ค่ายังเป็น −6 (ตัวลวง a คือ zero extension ซึ่งเปลี่ยนค่าเป็น +10)

N13. ตอบ c) carry out = 0, overflow = 1

0111 0000 + 0101 0000 = 1100 0000 ไม่มีบิตทดออก แต่บวก + บวก ได้ลบ (112 + 80 = 192 เกิน 127)

N14. ตอบ d) carry out = 1, overflow = 1

ได้ 1 0011 0000 → carry = 1 และผล 0x30 เป็นบวกจากลบ + ลบ ((−112) + (−96) = −208)

N15. ตอบ a) 0x7F + 0x01

127 + 1 = 128 เกินช่วง ข้อ b และ d บวกเลขต่างเครื่องหมาย ส่วนข้อ c (−1 + 1) มี carry แต่ไม่ overflow

N16. 1101 1011 = 0xDB

+37 = 0010 0101 → กลับบิตได้ 1101 1010 → +1 ได้ 1101 1011 (ตรวจ: 219 − 256 = −37)

N17. (ก) 1110 (ข) 0101 1001

- Gray: g = b $\oplus$ (b >> 1) = 1011 $\oplus$ 0101 = 1110
- BCD: 5 = 0101 และ 9 = 1001 (แต่ละหลักฐานสิบใช้ 4 บิต)

N18. มีศูนย์ตัวเดียว และใช้ adder ตัวเดียวทำได้ทั้งบวกและลบ

- sign-magnitude และ 1's complement มี +0 กับ −0 ทำให้ต้องตรวจกรณีพิเศษ
- การลบ $A - B = A + \bar{B} + 1$ จึงใช้วงจรบวกเดิมได้ (ALU ของ nanoLADA ก็ทำ SUB แบบนี้)
- ผลข้างเคียงคือช่วงไม่สมมาตร (8 บิต: −128 … 127)

DL2: Boolean Algebra และ K-map

สรุปก่อนทำโจทย์

- **DeMorgan:** (AB)' = A' + B' และ (A + B)' = A'B' (กลับตัวแปร + สลับ AND ↔ OR)
- **กฎลดรูปที่ใช้บ่อย:** A + AB = A · **A + A'B = A + B** · **AB + A'C + BC = AB + A'C** (consensus) · A + BC = (A + B)(A + C)
- **XOR** A ⊕ B = A'B + AB' (เป็น 1 เมื่อต่างกัน) · **XNOR** เป็น 1 เมื่อเท่ากัน
- **Universal gate:** NAND หรือ NOR อย่างเดียวก็สร้างได้ทุกฟังก์ชัน
- **Minterm m_i** = AND ที่เป็น 1 แถวเดียว (บิต 0 ใช้ตัวกลับ) · **Maxterm M_i** = OR ที่เป็น 0 แถวเดียว (บิต 1 ใช้ตัวกลับ)
- **Σm กับ ΠM** ของฟังก์ชันเดียวกันใช้เลขแถวที่เหลือกันพอดี
- **K-map:** แกนเรียงแบบ Gray (00, 01, 11, 10) ขอบซ้าย-ขวาและบน-ล่างติดกัน (มุมทั้งสี่ติดกัน) วงกลุ่มขนาด 2^k ให้ใหญ่และน้อยกลุ่มที่สุด และใช้ **don't care** เมื่อช่วยให้กลุ่มใหญ่ขึ้น

AB \ CD	00	01	11	10
00	0	1	3	2
01	4	5	7	6
11	12	13	15	14
10	8	9	11	10

โจทย์ (15 ข้อ)

ก. กฎของ Boolean

B1. ข้อใดถูกต้องตามกฎ DeMorgan

a) (AB)' = A'B'
b) (A + B)' = A' + B'
c) (AB)' = A' + B'
d) (A + B)' = AB

B2. A + A'B ลดรูปได้เป็น

a) A
b) A + B
c) AB
d) A'B

B3. A ⊕ B เท่ากับข้อใด

a) AB + A'B'
b) A'B + AB'
c) (A + B)'
d) A'B'

B4. AB + A'C + BC ลดรูปได้เป็น

a) AB + A'C
b) AB + BC
c) A + C
d) BC

B5. gate ชนิดใดเป็น universal gate

a) XOR
b) AND
c) NAND
d) OR

ข. Canonical form

B6. ใน 3 ตัวแปร ABC minterm m_5 คือ

a) A'BC'
b) AB'C
c) A + B' + C
d) ABC'

B7. ฟังก์ชัน 3 ตัวแปร F = Σm(0, 1, 2) เขียนในรูป POS ได้เป็น

a) ΠM(0, 1, 2)
b) ΠM(3, 4, 5, 6, 7)
c) ΠM(5, 6, 7)
d) Σm(3, 4, 5, 6, 7)

ค. ลดรูปด้วย K-map

B8. แกนของ K-map 2 ตัวแปรต้องเรียงลำดับอย่างไร

a) 00, 01, 10, 11
b) 00, 01, 11, 10
c) 11, 10, 01, 00
d) เรียงอย่างไรก็ได้

B9. F(A,B,C) = A'B'C' + A'B'C + AB'C' + AB'C ลดรูปได้เป็น

a) A'
b) B'
c) B'C'
d) A + B'

B10. F(A,B,C) = Σm(1, 3, 5, 7) ลดรูปได้เป็น

a) A
b) B'
c) C
d) A + C

B11. F(A,B,C) = Σm(0, 2, 4, 6) ลดรูปได้เป็น

a) A'
b) C'
c) B'
d) A'C'

B12. F(A,B,C,D) = Σm(0, 2, 8, 10) + d(1, 3, 9, 11) รูปที่ลดได้น้อยที่สุดคือ

a) B'D'
b) A'B'
c) B'
d) B' + D'

ง. ตอบสั้น

B13. ต้องใช้ 2-input NAND gate อย่างน้อยกี่ตัวจึงจะสร้าง A + B ได้ และเขียนนิพจน์ที่ใช้

B14. ลดรูป F(A,B,C,D) = Σm(0, 2, 5, 7, 8, 10, 13, 15)

B15. ลดรูป F(A,B,C,D) = Σm(4, 5, 6, 7, 12, 13)

เฉลย: DL2: Boolean Algebra และ K-map

ข้อ	B1	B2	B3	B4	B5	B6	B7	B8	B9	B10
ตอบ	c	b	b	a	c	b	b	b	b	c

ข้อ	B11	B12
ตอบ	b	c

B1. ตอบ c) (AB)' = A' + B'

กลับตัวแปรทุกตัวแล้วสลับ AND ↔ OR (a, b ลืมสลับเครื่องหมาย และ d ลืมกลับตัวแปร)

B2. ตอบ b) A + B

A + A'B = (A + A')(A + B) = A + B (ตัวลวง a คือ absorption ซึ่งใช้กับ A + AB)

B3. ตอบ b) A'B + AB'

XOR เป็น 1 เมื่ออินพุตต่างกัน (แถว 01 และ 10) ส่วนตัวลวง a คือ XNOR

B4. ตอบ a) AB + A'C

Consensus theorem: BC ถูกครอบคลุมด้วย AB (เมื่อ A = 1) และ A'C (เมื่อ A = 0) จึงตัดทิ้งได้

B5. ตอบ c) NAND

NAND (และ NOR) อย่างเดียวสร้าง NOT, AND, OR ได้ทั้งหมด ส่วน AND หรือ OR อย่างเดียวสร้าง NOT ไม่ได้

B6. ตอบ b) AB'C

5 = 101 → A = 1, B = 0, C = 1 → AB'C (ตัวลวง c คือ maxterm M_5 ที่ใช้บิตกลับด้าน)

B7. ตอบ b) ΠM(3, 4, 5, 6, 7)

POS ใช้แถวที่ F = 0 คือแถวที่เหลือ (ตัวลวง d คือ F')

B8. ตอบ b) 00, 01, 11, 10

เรียงแบบ Gray code เพื่อให้ช่องที่ติดกันต่างกันเพียง 1 ตัวแปร

B9. ตอบ b) B'

เป็น m(0,1,4,5) ซึ่งทุกตัวมี B = 0 ส่วน A และ C เปลี่ยนครบทุกค่า

B10. ตอบ c) C

001, 011, 101, 111 → C = 1 ทุกตัว

B11. ตอบ b) C'

000, 010, 100, 110 → C = 0 ทุกตัว

B12. ตอบ c) B'

4 มุมให้ B'D' แต่ถ้ารวม don't care จะได้ 8 ช่องที่ B = 0 ทั้งหมด → B' (ตัวลวง a ถูกแต่ยังไม่ minimal)

B13. 3 ตัว: A + B = NAND(NAND(A,A), NAND(B,B))

NAND(A,A) = A' และ NAND(B,B) = B' จากนั้น NAND(A', B') = (A'B')' = A + B (DeMorgan)

B14. F = B'D' + BD

m0, 2, 8, 10 คือ 4 มุม (B = 0, D = 0) และ m5, 7, 13, 15 คือสี่เหลี่ยมกลาง (B = 1, D = 1) → F = B ⊙ D (XNOR)

B15. F = A'B + BC'

- m4, 5, 6, 7 (แถว AB = 01 ทั้งแถว) → A'B
- m4, 5, 12, 13 (คอลัมน์ CD = 00, 01 ของแถว B = 1) → BC'
- สองกลุ่มซ้อนกันได้ และครอบคลุมทุก 1 แล้ว

DL3: Combinational Circuits

สรุปก่อนทำโจทย์

- **Combinational:** output ขึ้นกับ input ปัจจุบันเท่านั้น (ไม่มีหน่วยความจำ) เช่น MUX, decoder, adder, ALU
- **MUX 2^n-to-1** มี select **n** เส้น ใช้ "เลือก" ข้อมูลทุกจุดใน datapath (sel_b, sel_data, forwarding)
- **Decoder n-to-2^n:** output เส้นที่ i = minterm m_i (one-hot) ดังนั้น decoder + OR สร้าง SOP ใดก็ได้ และใช้เลือก register ที่จะเขียน
- **Half adder:** $S = A \oplus B$, $C = AB$ · **Full adder:** $S = A \oplus B \oplus Cin$, $Cout = AB + Cin(A \oplus B)$
- **Ripple-carry** ช้า (delay $\propto n$) ส่วน **carry-lookahead** ใช้ $g = AB$, $p = A \oplus B$ คำนวณ carry ขนาน เร็วกว่าแต่ใช้ gate มากกว่า
- **ALU:** การลบ $A - B = A + \bar{B} + 1$ และ **zero flag = NOR ของทุกบิตของผล**
- **ROM** เก็บ truth table ทั้งตาราง · **PLA** มี AND/OR plane ที่ program ได้ทั้งคู่ · **PAL** มี OR plane fixed

โจทย์ (12 ข้อ)

ก. แนวคิด

C1. ข้อใดคือนิยามของ combinational circuit

a) output ขึ้นกับ input ปัจจุบันเท่านั้น
b) output ขึ้นกับ state ที่จำไว้
c) ต้องมี clock เสมอ
d) มี flip-flop อย่างน้อย 1 ตัว

C2. Multiplexer แบบ 32-to-1 ต้องมี select lines กี่เส้น

a) 4
b) 5
c) 16
d) 32

C3. ต้องใช้ MUX แบบ 2-to-1 กี่ตัวจึงจะสร้าง MUX แบบ 8-to-1 ได้

a) 3
b) 4
c) 7
d) 8

C4. Decoder 3-to-8 ได้รับอินพุต ABC = 101 output เส้นใดเป็น 1

a) Y2
b) Y3
c) Y5
d) Y6

C5. Decoder n-to-2^n ร่วมกับ OR gate สามารถสร้างอะไรได้

a) เฉพาะฟังก์ชัน XOR
b) ฟังก์ชัน SOP ใดก็ได้ของ n ตัวแปร
c) flip-flop
d) เฉพาะ MUX

C6. Half adder ให้ output ใด

a) S = A $\oplus$ B, C = AB
b) S = A + B, C = AB
c) S = AB, C = A $\oplus$ B
d) S = A $\oplus$ B $\oplus$ Cin, C = AB

C7. ข้อใดถูกเกี่ยวกับ carry-lookahead adder เมื่อเทียบกับ ripple-carry adder

a) ช้ากว่าแต่ใช้ gate น้อยกว่า
b) เร็วกว่าแต่ใช้ gate มากกว่า
c) เร็วกว่าและใช้ gate น้อยกว่า
d) เหมือนกันทุกประการ

C8. Zero flag ของ ALU สร้างได้จาก gate ใด

a) AND ของทุกบิตของผล
b) OR ของทุกบิตของผล
c) NOR ของทุกบิตของผล
d) XOR ของทุกบิตของผล

C9. ข้อใดถูกเกี่ยวกับ PLA

a) AND plane fixed, OR plane program ได้
b) program ได้ทั้ง AND plane และ OR plane
c) เก็บ truth table ทุกแถวเหมือน ROM
d) ใช้ได้เฉพาะวงจร sequential

ข. ตอบสั้น

C10. เขียนสมการ Boolean ของ Sum และ Carry-out ของ full adder (อินพุต A, B, Cin)

C11. Ripple-carry adder ขนาด 16 bits ถ้า carry ผ่าน full adder แต่ละตัวใช้ 2 gate delays carry ออกจากบิตบนสุดจะช้าที่สุดกี่ gate delays

C12. ALU ทำ A − B ด้วย adder ได้อย่างไร

เฉลย: DL3: Combinational Circuits

ข้อ	C1	C2	C3	C4	C5	C6	C7	C8	C9
ตอบ	a	b	c	c	b	a	b	c	b

C1. ตอบ a) output ขึ้นกับ input ปัจจุบันเท่านั้น

b, c, d เป็นลักษณะของ sequential circuit

C2. ตอบ b) 5

$2^5 = 32$ (ตัวลวง d นับจำนวน data input)

C3. ตอบ c) 7

ต่อเป็นต้นไม้: 4 + 2 + 1 = 7 (ตัวลวง a คือจำนวน select lines)

C4. ตอบ c) Y5

$101_2 = 5$ และ Y5 = m_5 = AB'C

C5. ตอบ b) ฟังก์ชัน SOP ใดก็ได้ของ n ตัวแปร

output แต่ละเส้นของ decoder คือ minterm จึง OR minterm ที่ต้องการเข้าด้วยกันได้

C6. ตอบ a) S = A $\oplus$ B, C = AB

half adder ไม่มี carry-in (ตัวลวง d ผสม full adder เข้ามา)

C7. ตอบ b) เร็วกว่าแต่ใช้ gate มากกว่า

คำนวณ carry ทุกบิตขนานกันด้วย generate/propagate ซึ่งเป็น trade-off ระหว่าง area กับ speed

C8. ตอบ c) NOR ของทุกบิตของผล

เป็น 1 เมื่อทุกบิตเป็น 0 ซึ่ง BEQ ใน nanoLADA ใช้ zero flag หลังทำ SUB

C9. ตอบ b) program ได้ทั้ง AND plane และ OR plane

ตัวลวง a คือ PAL กลับด้าน (PAL program ได้แค่ AND plane) และตัวลวง c คือ ROM

C10. **Sum = A ⊕ B ⊕ Cin, Cout = AB + Cin(A ⊕ B)**

Cout เขียนแบบ majority ได้เป็น AB + ACin + BCin

C11. **32 gate delays**

carry ต้องไหลผ่านครบ 16 ตัว: 16 × 2 = 32 ซึ่งเป็นเหตุผลที่ต้องมี carry-lookahead

C12. **A + $\bar{B}$ + 1: กลับทุกบิตของ B และให้ Cin = 1**

เป็นการบวก −B ในรูป 2's complement จึงใช้ adder ตัวเดียวทำได้ทั้งบวกและลบ

DL4: Sequential Circuits และ FSM

สรุปก่อนทำโจทย์

- **Latch** level-sensitive · **Flip-flop** edge-triggered
- **D:** $Q^+ = D$ · **JK:** $Q^+ = JQ' + K'Q$ (00 hold, 01 reset, 10 set, **11 toggle**) · **T:** $Q^+ = T \oplus Q$ · **SR:** S = R = 1 ห้ามใช้

Q → Q⁺	D	T	J K
0 → 0	0	0	0 X
0 → 1	1	1	1 X
1 → 0	0	1	X 1
1 → 1	1	0	X 0

- **Setup** = ต้องนิ่งก่อนขอบ clock · **Hold** = ต้องนิ่งหลังขอบ · $\mathbf{T_{clk} \geq t_{clk\text{-}to\text{-}Q} + t_{comb,max} + t_{setup}}$
- **Counter modulo-N** ใช้ FF $\lceil \log_2 N \rceil$ ตัว
- **FSM:** Moore output ขึ้นกับ state อย่างเดียว (เขียนในวงกลม) ส่วน Mealy ขึ้นกับ state + input (เขียนบนเส้น) และ Mealy มักใช้ state น้อยกว่า
- **FF ขั้นต่ำ = $\lceil \log_2(\#states) \rceil$** (binary) ส่วน one-hot ใช้เท่าจำนวน state
- control unit ของ multi-cycle processor คือ FSM (Ch5)

โจทย์ (14 ข้อ)

ก. Flip-flop

F1. อุปกรณ์ใดเปลี่ยนค่าเฉพาะที่ขอบ (edge) ของ clock

a) SR latch
b) D latch
c) D flip-flop
d) Combinational circuit

F2. Characteristic equation ของ D flip-flop คือ

a) $Q^+ = D$
b) $Q^+ = D \oplus Q$
c) $Q^+ = DQ'$
d) $Q^+ = D'$

F3. JK flip-flop ที่มี J = 1 และ K = 1 จะเกิดอะไรขึ้นที่ขอบ clock

a) Q = 0
b) Q = 1
c) Q คงเดิม
d) Q กลับค่า (toggle)

F4. T flip-flop มี Q = 1 และ T = 1 หลังขอบ clock ค่า Q คือ

a) 0
b) 1
c) คงเดิม
d) ไม่แน่นอน

F5. SR latch มีสถานะต้องห้ามเมื่อใด

a) S = 0, R = 0
b) S = 1, R = 0
c) S = 0, R = 1
d) S = 1, R = 1

F6. Setup time หมายถึงข้อใด

a) เวลาที่ input ต้องนิ่งก่อนขอบ clock
b) เวลาที่ input ต้องนิ่งหลังขอบ clock
c) เวลาตั้งแต่ขอบ clock จนถึง output เปลี่ยน
d) ความยาวของ clock cycle

ข. FSM

F7. ข้อใดถูกต้องเกี่ยวกับ Moore machine

a) output ขึ้นกับ state ปัจจุบันเท่านั้น
b) output ขึ้นกับ input ปัจจุบันเท่านั้น
c) ใช้ state น้อยกว่า Mealy เสมอ
d) output เขียนไว้บนเส้น transition

F8. Mealy machine ที่ตรวจจับลำดับ "101" แบบ overlapping ต้องใช้อย่างน้อยกี่ state

a) 2
b) 3
c) 4
d) 5

F9. FSM ที่มี 10 states ถ้าใช้ binary encoding ต้องใช้ flip-flop อย่างน้อยกี่ตัว

a) 3
b) 4
c) 5
d) 10

F10. FSM ที่มี 12 states ถ้าใช้ one-hot encoding ต้องใช้ flip-flop กี่ตัว

a) 4
b) 6
c) 12
d) 24

ค. ตอบสั้น

F11. ต้องการให้ Q เปลี่ยนจาก 0 เป็น 1 ต้องป้อนอะไรให้ (ก) JK flip-flop (ข) T flip-flop

---

F12. ออกแบบ Moore machine ตรวจจับ "101" แบบ overlapping ต้องใช้กี่ state และ flip-flop อย่างน้อยกี่ตัว

F13. วงจรมี $t_{clk\text{-}to\text{-}Q}$ = 2 ns, combinational delay สูงสุด 10 ns และ setup time 1 ns clock period ขั้นต่ำและความถี่สูงสุดคือเท่าใด

F14. Counter แบบ modulo-12 ต้องใช้ flip-flop อย่างน้อยกี่ตัว

เฉลย: DL4: Sequential Circuits และ FSM

ข้อ	F1	F2	F3	F4	F5	F6	F7	F8	F9	F10
ตอบ	c	a	d	a	d	a	a	b	b	c

F1. ตอบ c) D flip-flop

latch เป็น level-sensitive (โปร่งใสตลอดช่วงที่ enable)

F2. ตอบ a) $Q^+ = D$

D flip-flop เก็บค่า D ณ ขอบ clock จึงนำไปทำ register

F3. ตอบ d) Q กลับค่า (toggle)

$Q^+ = JQ' + K'Q = Q'$ (สถานะต้องห้ามเป็นของ SR)

F4. ตอบ a) 0

$Q^+ = T \oplus Q = 1 \oplus 1 = 0$

F5. ตอบ d) S = 1, R = 1

แบบ NOR ทำให้ Q และ Q' เป็น 0 พร้อมกัน แล้วค่าไม่แน่นอนเมื่อปล่อย

F6. ตอบ a) เวลาที่ input ต้องนิ่งก่อนขอบ clock

b คือ hold time และ c คือ clk-to-Q

F7. ตอบ a) output ขึ้นกับ state ปัจจุบันเท่านั้น

d เป็นลักษณะของ Mealy และ Moore มักใช้ state มากกว่า

F8. ตอบ b) 3

S0, S1 ("1") และ S2 ("10") โดย output = 1 บนเส้น S2 →(1)→ S1

F9. ตอบ b) 4

$\lceil \log_2 10 \rceil = 4$ ($2^3 = 8$ ไม่พอ)

F10. ตอบ c) 12

one-hot ใช้ 1 FF ต่อ 1 state (ตัวลวง a คือ binary encoding)

F11. (ก) J = 1, K = X (ข) T = 1

JK เปลี่ยน 0 → 1 ได้ทั้ง set (10) และ toggle (11) จึงไม่สน K

F12. 4 states, 2 flip-flops

S0 → S1 ("1") → S2 ("10") → S3 ("101", output 1) จาก S3 ถ้าได้ 0 ไป S2 และถ้าได้ 1 ไป S1

F13. 13 ns ≈ 76.9 MHz

T ≥ 2 + 10 + 1 = 13 ns → f = 1/13 ns ≈ 76.9 MHz (เหตุผลเดียวกับที่ pipeline register มี overhead)

F14. 4 ตัว

$\lceil \log_2 12 \rceil = 4$

Ch1: Introduction (ISA vs Organization)

สรุปก่อนทำโจทย์

- **ISA** = interface/สัญญาระหว่าง software กับ hardware กำหนด instructions, registers, addressing, format และ interrupt/exception ซึ่งเป็น abstraction ได้ 2 มุม: of machine for software และ **of software for hardware** (โปรแกรม 8088 ยังรันบน Core i7 ได้)
- **Organization** = โครงสร้างภายในที่ implement ISA: ALU, Control Unit, Register, Bus, Memory subsystem, I/O subsystem → **i3/i5/i7 ต่างกันที่ organization** (core, cache, Turbo Boost, Hyper-Threading)
- **ABI** = ข้อตกลงระดับ binary: calling convention, parameter passing, stack frame, alignment
- **Abstraction layers:** Application → OS/Compiler → **ISA** → Firmware → Processor/I/O → Datapath & Control → Digital design → Circuit → Layout
- **Trade-off หลัก:** Cost vs Performance (vs Power) ต้องใช้ modeling + measurement และออกแบบแบบ iterative
- **Trends:** Moore's Law (transistor ×2 ทุก ~2 ปี/18 เดือน, เป็นข้อสังเกต) · DRAM capacity ×4 ทุก 3 ปี · ราคา DRAM ลด 40% ต่อปี · software ใช้ memory ×2 ต่อปี
- **IBM 360 (1964)** รวม ISA 7 แบบ ทำให้ software portable

โจทย์ (11 ข้อ)

ก. Choice

I1. Intel เพิ่ม L3 cache และ Hyper-Threading โดยโปรแกรมเดิมรันได้ทันทีไม่ต้อง compile ใหม่ นี่คือการเปลี่ยนแปลงในระดับใด

a) ISA
b) ABI
c) Computer Organization
d) Compiler

I2. การที่ซอฟต์แวร์ของ 8088 ยังรันบน Core i7 ได้ แสดงถึงข้อใด

a) Moore's Law
b) ISA เป็น abstraction of software for hardware (backward compatibility)
c) Core i7 มี organization เหมือน 8088
d) Amdahl's Law

I3. ข้อใดไม่ใช่ส่วนประกอบของ Computer Organization ตามสไลด์

a) ALU
b) Bus
c) Control Unit
d) Instruction format

I4. ข้อใดไม่ใช่สิ่งที่ ISA กำหนด

a) รูปแบบของคำสั่ง
b) addressing modes
c) จำนวน cache level ใน CPU
d) วิธีจัดการ interrupt/exception

I5. ในลำดับชั้นของ abstraction ชั้นใดอยู่ระหว่าง compiler/OS กับ hardware

a) ABI
b) ISA
c) Datapath
d) Layout

I6. Moore's Law กล่าวว่า

a) clock rate เพิ่มเป็น 2 เท่าทุก 2 ปี
b) จำนวน transistor บน IC เพิ่มเป็น 2 เท่าประมาณทุก 2 ปี (18 เดือน)
c) ราคา DRAM ลดลง 40% ต่อปี
d) performance เพิ่มตามจำนวน core

I7. ตามสไลด์ trade-off หลักของการออกแบบคอมพิวเตอร์คือ

a) Cost vs Performance (vs Power)
b) Latency vs Bandwidth เท่านั้น
c) RISC vs CISC
d) Hardware vs Firmware

I8. ข้อใดถูกตาม trend ในสไลด์

a) DRAM ความเร็วเพิ่ม 4 เท่าทุก 3 ปี
b) DRAM ความจุเพิ่ม 4 เท่าทุก 3 ปี แต่ความเร็วเพิ่มช้า
c) Disk ความจุคงที่
d) ราคา DRAM เพิ่มขึ้นทุกปี

ข. ตอบสั้น

I9. เติมคำ: ISA เป็น interface ระหว่าง ..... กับ ..... และในปี 1964 IBM ออก ..... ซึ่งรวม instruction set ..... แบบเข้าด้วยกัน

I10. ABI กำหนดอะไรบ้าง และต่างจาก ISA อย่างไร (ตอบสั้น)

I11. ทำไม CPU ต้องมี register (เปรียบเทียบตามสไลด์)

เฉลย: Ch1: Introduction (ISA vs Organization)

ข้อ	I1	I2	I3	I4	I5	I6	I7	I8
ตอบ	c	b	d	c	b	b	a	b

I1. ตอบ c) Computer Organization

binary เดิมรันได้แปลว่า ISA ไม่เปลี่ยน ส่วน cache และ HT เป็นโครงสร้างภายใน

I2. ตอบ b) ISA เป็น abstraction of software for hardware (backward compatibility)

organization ต่างกันมาก มีแค่ ISA ที่ compatible กัน

I3. ตอบ d) Instruction format

instruction format เป็นส่วนของ ISA

I4. ตอบ c) จำนวน cache level ใน CPU

cache เป็น organization ซึ่ง software มองไม่เห็นผ่าน ISA

I5. ตอบ b) ISA

ISA คือ interface ระหว่าง software กับ hardware

I6. ตอบ b) จำนวน transistor บน IC เพิ่มเป็น 2 เท่าประมาณทุก 2 ปี (18 เดือน)

เป็นข้อสังเกตของ Gordon Moore เรื่องจำนวน transistor ไม่ได้พูดถึง clock หรือ performance

I7. ตอบ a) Cost vs Performance (vs Power)

และต้องใช้ modeling กับ measurement ในการตัดสินใจ

I8. ตอบ b) DRAM ความจุเพิ่ม 4 เท่าทุก 3 ปี แต่ความเร็วเพิ่มช้า

ความจุโตเร็ว แต่ performance ของ DRAM ดีขึ้นช้า จึงเป็นที่มาของ memory hierarchy (Ch8)

I9. software, hardware, IBM 360 (System/360), 7

ก่อนหน้านั้นโปรแกรมไม่ portable

I10. ABI กำหนดข้อตกลงระดับ binary เช่น calling convention, การส่ง parameter (register/stack), caller/callee-save, stack frame, alignment

ISA บอกว่ามีคำสั่งอะไร (เช่นมี CALL) ส่วน ABI บอกว่า binary ที่ compile แยกกันต้องใช้คำสั่งเหล่านั้นร่วมกันอย่างไร

I11. เป็นที่เก็บชั่วคราวที่เร็วที่สุด ใกล้ ALU ทำให้ไม่ต้องไป memory ทุกครั้ง

สไลด์เปรียบกับการบวก 2 + 3 แบบประถม คือ "จำ 2 ไว้ในหัว" แล้วนับต่อ และ register เป็นชั้นบนสุดของ memory hierarchy

Ch2: Performance (Speedup, CPU Time, Amdahl)

สรุปก่อนทำโจทย์

```
Speedup ("X เร็วกว่า Y n เท่า") = T_Y / T_X = T_old / T_new
CPU Time = IC × CPI × T_c = IC × CPI / f
CPI_avg = Σ(CPI_i × freq_i) · MIPS = IC / (T × 10^6) = f / (CPI × 10^6)
Amdahl: S = 1 / ((1 - P) + P/n) · S_max = 1 / (1 - P) · P = สัดส่วนของ "เวลา"
```

- **Response time** (latency, elapsed/wall-clock หรือ CPU time) กับ **throughput** (งานต่อเวลา) เป็นคนละเรื่องกัน และ `time` ให้ real = elapsed, user, sys
- **Benchmark:** real · microbenchmark · kernel · **toy** (quicksort) · **synthetic** (Whetstone, Dhrystone) · SPEC
- **Weighted AM** ใช้กับเวลาเมื่อรู้ workload · **Geometric mean** ใช้กับ ratio (consistent กับทุก baseline แต่ทำนายเวลาจริงไม่ได้)
- **IC** ขึ้นกับ program/ISA/compiler · **CPI** ขึ้นกับ ISA/organization · $\mathbf{T_c}$ ขึ้นกับ organization/VLSI technology
- **MIPS ใช้เทียบข้าม ISA ไม่ได้** และ clock สูงกว่าไม่ได้แปลว่าเร็วกว่า

โจทย์ (30 ข้อ)

ก. นิยามและการวัด

P1. Computer A ใช้เวลา 15 s ส่วน Computer B ใช้ 24 s กับโปรแกรมเดียวกัน ข้อใดถูก

a) A เร็วกว่า B 1.6 เท่า
b) A เร็วกว่า B 0.625 เท่า
c) B เร็วกว่า A 1.6 เท่า
d) A เร็วกว่า B 9 เท่า

P2. โปรแกรมเดิมใช้เวลา 10 s หลังปรับปรุงเหลือ 4 s ได้ speedup เท่าใด

a) 0.4
b) 1.5
c) 2.5
d) 6

P3. X ใช้ 30 s และ Y ใช้ 20 s กับโปรแกรมเดียวกัน Y เร็วกว่า X กี่เท่า

a) 0.67
b) 1.5
c) 10
d) 50

P4. Computer X ทำ 4 งานพร้อมกันเสร็จใน 8 นาที ส่วน Y ทำทีละงาน งานละ 3 นาที ข้อใดถูก

a) X มี throughput สูงกว่า และ Y มี response time ต่ำกว่า
b) Y ดีกว่าทั้งสองด้าน
c) X ดีกว่าทั้งสองด้าน
d) throughput เท่ากัน

P5. ในผลลัพธ์ของคำสั่ง `time` ค่า real คือ

a) CPU time ใน user mode
b) CPU time ใน kernel
c) Elapsed (wall-clock) time
d) CPU time ของทุก process

P6. การใช้ quicksort สั้น ๆ เป็นตัววัด performance คือ benchmark ประเภทใด

a) Real
b) Kernel
c) Synthetic
d) Toy

P7. Whetstone และ Dhrystone เป็น benchmark ประเภทใด

a) Real
b) Toy
c) Synthetic
d) Microbenchmark

ข. การรวมผล

P8. ถ้ารู้สัดส่วนการใช้งานของแต่ละโปรแกรม และต้องการรวม "เวลา" เป็นค่าเดียว ควรใช้ค่าเฉลี่ยแบบใด

a) Weighted arithmetic mean
b) Geometric mean
c) Harmonic mean
d) Median

P9. เหตุผลที่ใช้ geometric mean กับผล benchmark ที่เป็น ratio คือ

a) ได้ค่ามากที่สุด
b) ให้ผลสอดคล้องกันไม่ว่าจะใช้เครื่องใดเป็น baseline
c) ทำนายเวลาจริงได้แม่นยำ
d) ไม่ต้องใช้ workload

P10. Speedup ของสองโปรแกรมคือ 2 และ 8 geometric mean คือ

a) 4
b) 5
c) 16
d) 3.2

ค. CPU Time และ MIPS

P11. โปรแกรมมี 4×10^9 คำสั่ง CPI 1.5 บน CPU 3 GHz ใช้ CPU time เท่าใด

a) 0.5 s
b) 1.125 s
c) 2 s
d) 8 s

P12. CPU 500 MHz รัน 2×10^8 คำสั่ง CPI 2.5 ใช้ CPU time เท่าใด

a) 0.1 s
b) 0.25 s
c) 1 s
d) 2.5 s

P13. เครื่องในข้อที่แล้วมีกี่ MIPS

a) 80
b) 200
c) 500
d) 1250

P14. โปรแกรมใช้ 3 × 10^9 cycles บน CPU 1.5 GHz ใช้เวลาเท่าใด

a) 0.5 s
b) 3 s
c) 2 s
d) 4.5 s

P15. Instruction mix: 60% มี CPI 1, 30% มี CPI 2 และ 10% มี CPI 5 average CPI คือ

a) 1.7
b) 2.67
c) 8
d) 1.5

P16. A มี 400 MIPS และรันโปรแกรมที่ compile ได้ 800 ล้านคำสั่ง ส่วน B (ISA ต่างกัน) มี 300 MIPS และโปรแกรมเดียวกัน compile ได้ 500 ล้านคำสั่ง ข้อใดถูก

a) A เร็วกว่าเพราะ MIPS สูงกว่า
b) B เร็วกว่าประมาณ 1.2 เท่า
c) เร็วเท่ากัน
d) สรุปไม่ได้เพราะไม่รู้ clock

P17. ตัวแปรใดใน CPU time ที่ขึ้นกับ VLSI technology โดยตรง

a) IC
b) CPI
c) Clock cycle time
d) จำนวนบรรทัดของโปรแกรม

ง. Amdahl's Law

P18. ส่วนที่กินเวลา 60% ถูกทำให้เร็วขึ้น 3 เท่า overall speedup คือ

a) 1.67
b) 1.8
c) 2.0
d) 3.0

P19. ทำให้ส่วนที่กินเวลา 50% เร็วขึ้น 2 เท่า จะได้ overall speedup เท่าใด

a) 1.33
b) 1.5
c) 2
d) 1.25

P20. โปรแกรมที่ทำ parallel ได้ 75% ของเวลา จะได้ speedup สูงสุดเท่าใด

a) 1.33
b) 3
c) 4
d) ไม่มีขีดจำกัด

จ. ตอบสั้น

P21. โปรแกรม 2 × 10^9 คำสั่ง บน CPU 2 GHz: ALU 40% (CPI 1), Load 30% (CPI 4), Store 10% (CPI 3), Branch 20% (CPI 2) จงหา average CPI และ CPU time

---

P22. 6 × 10^9 คำสั่ง CPI 1.25 บน CPU 2.5 GHz จงหา CPU time และ MIPS

P23. A และ B ใช้ ISA เดียวกัน โดย A: 2 GHz, CPI 2.0 และ B: 3 GHz, CPI 2.7 เครื่องใดเร็วกว่าและกี่เท่า

P24. (ตัวอย่างในสไลด์) CPU 1 GHz รัน 2000 ล้านคำสั่งใน 20 s ถ้าเพิ่ม clock เป็น 2 GHz แต่ CPI แย่ลง 1.2 เท่า จะใช้เวลาเท่าใด

P25. P1 ใช้ 70% และ P2 ใช้ 30% ของงาน เวลาที่วัดได้คือ A: P1 = 8 s, P2 = 20 s และ B: P1 = 12 s, P2 = 6 s จงหา weighted mean และบอกว่าควรเลือกเครื่องใด

P26. FP กินเวลา 40% ของโปรแกรม ทำให้ FP เร็วขึ้น 5 เท่า overall speedup เป็นเท่าใด

P27. ส่วนที่ปรับปรุงได้กินเวลา 90% ต้องเร่งส่วนนั้นกี่เท่าจึงจะได้ overall speedup = 5

P28. (Exercise ในสไลด์) FP 25% ของคำสั่ง CPI 4.0 ส่วนคำสั่งอื่น CPI 1.33 ถ้าลด CPI ของ FP เหลือ 2.5 จะได้ speedup เท่าใด

P29. 10^9 คำสั่ง บน 2.5 GHz: ALU 50% (CPI 1), Load 20% (CPI 5), Store 10% (CPI 4), Branch 20% (CPI 3) ถ้าลด CPI ของ Load เหลือ 3 จะได้ speedup เท่าใด และ P ใน Amdahl คือเท่าใด

P30. โปรแกรมทำ parallel ได้ 70% (ก) บน 8 cores ได้ speedup เท่าใด (ข) ระหว่างเพิ่มเป็น 64 cores กับทำส่วน serial ให้เร็วขึ้น 2 เท่า (8 cores เท่าเดิม) ทางไหนดีกว่า

เฉลย: Ch2: Performance (Speedup, CPU Time, Amdahl)

ข้อ	P1	P2	P3	P4	P5	P6	P7	P8	P9	P10
ตอบ	a	c	b	a	c	d	c	a	b	a

ข้อ	P11	P12	P13	P14	P15	P16	P17	P18	P19	P20
ตอบ	c	c	b	c	a	b	c	a	a	c

P1. ตอบ a) A เร็วกว่า B 1.6 เท่า

24 / 15 = 1.6 (b กลับเศษส่วน และ d เป็นผลต่าง)

P2. ตอบ c) 2.5

$T_{old} / T_{new} = 10/4$

P3. ตอบ b) 1.5

30/20 = 1.5

P4. ตอบ a) X มี throughput สูงกว่า และ Y มี response time ต่ำกว่า

X มี throughput 0.5 งาน/นาที ส่วน Y มี 0.33 งาน/นาที แต่ Y ใช้เวลา 3 นาทีต่องานเทียบกับ 8 นาทีของ X

P5. ตอบ c) Elapsed (wall-clock) time

user และ sys เป็น CPU time

P6. ตอบ d) Toy

synthetic คือโปรแกรมที่เขียนขึ้นให้เลียนแบบโปรแกรมจริง เช่น Whetstone และ Dhrystone

P7. ตอบ c) Synthetic

เขียนขึ้นเพื่อเลียนแบบพฤติกรรมของโปรแกรมจริง (scientific/integer)

P8. ตอบ a) Weighted arithmetic mean

เวลามีหน่วย ส่วน GM ใช้กับ ratio

P9. ตอบ b) ให้ผลสอดคล้องกันไม่ว่าจะใช้เครื่องใดเป็น baseline

สไลด์ระบุชัดว่า GM ทำนาย performance จริงไม่ได้ (ตัวลวง c)

P10. ตอบ a) 4

$\sqrt{(2 \times 8)} = 4$ (b เป็น arithmetic mean และ d เป็น harmonic mean)

P11. ตอบ c) 2 s

$4 \times 10^9 \times 1.5 / 3 \times 10^9 = 2$ s

P12. ตอบ c) 1 s

5×10^8 cycles / 5×10^8 Hz = 1 s

P13. ตอบ b) 200

500 / 2.5 = 200 (หรือ 2×10^8 คำสั่ง / 1 s)

P14. ตอบ c) 2 s

cycles / f = 2 s

P15. ตอบ a) 1.7

0.6 + 0.6 + 0.5 (ตัวลวง b เฉลี่ยแบบไม่ถ่วงน้ำหนัก)

P16. ตอบ b) B เร็วกว่าประมาณ 1.2 เท่า

$T_A = 2$ s และ $T_B = 1.67$ s → MIPS ใช้เทียบข้าม ISA ไม่ได้

P17. ตอบ c) Clock cycle time

T_c ขึ้นกับ organization และ VLSI ส่วน IC ขึ้นกับ program/ISA/compiler

P18. ตอบ a) 1.67

1 / (0.4 + 0.2) = 1.67

P19. ตอบ a) 1.33

1 / (0.5 + 0.25)

P20. ตอบ c) 4

1 / 0.25 = 4 เพราะส่วน serial จำกัด speedup

P21. CPI = 2.3, CPU time = 2.3 s

CPI = 0.4 + 1.2 + 0.3 + 0.4 = 2.3 และ T = $2 \times 10^9 \times 2.3 / 2 \times 10^9$ = 2.3 s (load มี 30% ของคำสั่งแต่กินเวลา 52%)

P22. 3 s, 2000 MIPS

T = $7.5 \times 10^9 / 2.5 \times 10^9$ = 3 s และ MIPS = 2500 / 1.25 = 2000

P23. B เร็วกว่า 1.11 เท่า

เวลาต่อคำสั่ง: A = 2.0/2 = 1.0 ns และ B = 2.7/3 = 0.9 ns → clock สูงขึ้น 1.5 เท่าแต่ได้แค่ 1.11 เท่า

P24. 12 s

$CPI_{เดิม}$ = $20 \times 10^9 / 2 \times 10^9$ = 10 → $CPI_{ใหม่}$ = 12 → T = $2 \times 10^9 \times 12 / 2 \times 10^9$ = 12 s

P25. A = 11.6 s, B = 10.2 s → เลือก B

A = 5.6 + 6.0 และ B = 8.4 + 1.8 → "เร็วที่สุด" ขึ้นกับ workload

P26. 1.47

1 / (0.6 + 0.08) = 1/0.68

P27. 9 เท่า

1/(0.1 + 0.9/n) = 5 → 0.9/n = 0.1 → n = 9 (S_{max} = 10)

P28. ≈ 1.23

$CPI_{เดิม}$ = 1 + 0.9975 ≈ 2.0 และ $CPI_{ใหม่}$ = 0.625 + 0.9975 = 1.6225 → 2.0/1.6225 ≈ 1.23 (ถ้าใช้ Amdahl ต้องแทน P = เวลาของ FP = 1.0/2.0 = 50% ไม่ใช่ 25%)

P29. speedup ≈ 1.19, P = 0.4

CPI 2.5 → 2.1 ได้ 2.5/2.1 = 1.19 · load กินเวลา 1.0/2.5 = 40% → 1/(0.6 + 0.4 × 3/5) = 1/0.84 = 1.19

P30. (ก) 2.58 (ข) ทำ serial เร็วขึ้นดีกว่า (4.21 > 3.22)

- 64 cores: 1/(0.3 + 0.7/64) = 3.22
- serial เร็วขึ้น 2 เท่า: 1/(0.15 + 0.0875) = 4.21
- → ส่วน serial เป็นตัวจำกัด (diminishing returns)

Ch3 (1): ISA Types, CISC/RISC, Format และ Compiler

สรุปก่อนทำโจทย์

ISA (x = (a·b) + (a – (b·c)))	#address	#คำสั่ง	จุดเด่น	จุดด้อย
Accumulator	1	11	HW น้อยที่สุด	ACC เป็น bottleneck, memory traffic สูง
Stack	0	10	code density ดี, compiler ง่าย	data ไม่อยู่ top เสมอ, optimize ยาก
Memory-Memory	2–3	4 (3-op) / 7 (2-op)	คำสั่งน้อยที่สุด	memory traffic สูงมาก, CPI แปรผัน
Register-Memory (x86)	2	10	ไม่ต้อง load ก่อน	ไม่ orthogonal, CPI แปรผัน
Load-Store (RISC)	3	8	fixed length, CPI ใกล้กัน, **pipeline ง่าย**	IC สูง, ต้องพึ่ง compiler

- **ISA ที่ดี:** compatibility, generality, **orthogonality** (ทุก mode ใช้กับทุก type/instruction ได้), regularity, ease of compilation/implementation
- **CISC vs RISC:** เมื่อมี ILP RISC ชนะ · **Post-RISC:** x86 แปลเป็น micro-ops ภายใน · ARM = Acorn RISC Machine
- **Format:** variable length (code เล็ก decode ยาก) vs **fixed length (ชนะในแง่ performance)** · MIPS code ใหญ่ 2 เท่าของ VAX แต่ CPI เป็น 1/6 → เร็วกว่า 3 เท่า
- **Compiler optimization:** inlining · local: CSE, constant propagation · global: copy propagation, **code motion** · machine-dependent: **strength reduction**, pipeline scheduling
- **Basic block** = ไม่มี branch เข้าออกตรงกลาง · **alias (pointer)** ทำให้เก็บตัวแปรใน register ไม่ได้ · bug ส่วนใหญ่ของ compiler มาจาก optimization · **XOR R1,R1,R1** ใช้ clear register

โจทย์ (19 ข้อ)

ก. ISA 5 แบบ

A1. สถาปัตยกรรมแบบใดเป็น "0-address" (operand อยู่ที่ top of stack โดยปริยาย)

a) Accumulator
b) Stack
c) Register-Memory
d) Load-Store

A2. สถาปัตยกรรมใดคำนวณตัวอย่างในสไลด์ได้ด้วยคำสั่งน้อยที่สุด (4 คำสั่ง)

a) Memory-Memory (3 operands)
b) Load-Store
c) Stack
d) Accumulator

A3. สถาปัตยกรรมใดเหมาะกับ pipelining ที่สุด

a) Accumulator
b) Stack
c) Memory-Memory
d) Register-Register (Load-Store)

A4. ข้อใดเป็นข้อเสียของ Stack architecture

a) code density แย่
b) ข้อมูลที่ต้องการไม่ได้อยู่ top of stack เสมอ
c) ต้องใช้ hardware มาก
d) compiler เขียนยาก

A5. ข้อใดเป็นข้อเสียของ Accumulator architecture

a) ต้องใช้ register จำนวนมาก
b) decode ซับซ้อน
c) code density แย่ที่สุด
d) memory traffic สูง และ accumulator เป็น bottleneck

A6. ข้อเสียของ Register-Memory architecture ตามสไลด์คือ

a) ต้อง load ทุก operand ก่อนใช้
b) poor orthogonality (operand สองตัวมีสิทธิ์ไม่เท่ากัน) และ CPI แปรผัน
c) ไม่มี register เลย
d) ใช้ได้แค่ 0-address

A7. Orthogonality ของ ISA หมายถึงข้อใด

a) คำสั่งทุกตัวยาวเท่ากัน
b) มี special case น้อย ใช้ทุก addressing mode กับทุก data type/instruction ได้
c) มี register 32 ตัว
d) ไม่มี immediate

ข. CISC/RISC และ Instruction Format

A8. ข้อใดอธิบายยุค Post-RISC ได้ถูกต้อง

a) x86 แปลคำสั่ง CISC เป็นคำสั่งภายในคล้าย RISC (micro-ops) ก่อนทำ parallelism
b) RISC ถูกเลิกใช้แล้ว
c) ARM กลายเป็น CISC
d) x86 เปลี่ยนเป็น fixed-length

A9. ARM ย่อมาจาก

a) Advanced Register Machine
b) Acorn RISC Machine
c) Accumulator RISC Model
d) Array Register Memory

A10. ทำไม fixed-length instruction จึงชนะในแง่ performance

a) code เล็กที่สุดเสมอ
b) decode ง่ายและทำ pipeline ได้ง่าย
c) มี addressing mode มากที่สุด
d) ไม่ต้องมี opcode

A11. code ของ MIPS ใหญ่ประมาณ 2 เท่าของ VAX แต่ CPI เป็น 1/6 ของ VAX ถ้า clock เท่ากัน ข้อใดถูก

a) VAX เร็วกว่า 2 เท่า
b) MIPS เร็วกว่าประมาณ 3 เท่า
c) MIPS เร็วกว่า 6 เท่า
d) เร็วเท่ากัน

ค. Compiler

A12. วิธี clear register ที่สไลด์แนะนำสำหรับ architecture ส่วนใหญ่คือ

a) `CLR R1`
b) `MOV R1, 0`
c) `SUB R1,R1,R1`
d) `XOR R1,R1,R1`

A13. การย้าย `hs = 3600;` ที่เป็นค่าคงที่ออกไปนอก loop คือ optimization แบบใด

a) Strength reduction
b) Code motion
c) Procedure inlining
d) Pipeline scheduling

A14. เปลี่ยน `x * 8` เป็น `x << 3` คือ optimization แบบใด

a) Code motion
b) Strength reduction
c) Common subexpression elimination
d) Inlining

A15. Basic block คืออะไร

a) ลำดับคำสั่งที่ไม่มี branch เข้าหรือออกตรงกลาง
b) block ของ cache
c) ฟังก์ชันหนึ่งฟังก์ชัน
d) หน่วยของ page

A16. อะไรทำให้ compiler เก็บตัวแปรไว้ใน register ไม่ได้ ตามสไลด์

a) Loop
b) Alias (pointer)
c) Constant
d) Comment

ง. ตอบสั้น

A17. เติมจำนวนคำสั่งที่ใช้คำนวณ x = (a·b) + (a – (b·c)) ของแต่ละ ISA: Accumulator ....., Stack ....., Mem-Mem 3-op ....., Reg-Mem ....., Load-Store .....

A18. ใช้ copy propagation กับ `A=B+C; B=A+C; D=A-B;` จนไม่มี operand ที่เป็นค่าที่คำนวณได้ แล้วสรุปว่าได้บทเรียนอะไร

A19. optimize loop นี้ `for(x=0;x⩽100;x++){ hs=3600; ds=hs*24; h=a[x]/hs; d=a[x]/ds; }`

เฉลย: Ch3 (1): ISA Types, CISC/RISC, Format และ Compiler

ข้อ	A1	A2	A3	A4	A5	A6	A7	A8	A9	A10
ตอบ	b	a	d	b	d	b	b	a	b	b

ข้อ	A11	A12	A13	A14	A15	A16
ตอบ	b	d	b	b	a	b

A1. ตอบ b) Stack

Accumulator = 1, Register-Memory = 2, Load-Store = 3 address

A2. ตอบ a) Memory-Memory (3 operands)

แลกกับ memory traffic สูงมาก

A3. ตอบ d) Register-Register (Load-Store)

fixed-length และ CPI ใกล้เคียงกันทุกคำสั่ง

A4. ตอบ b) ข้อมูลที่ต้องการไม่ได้อยู่ top of stack เสมอ

a, c, d เป็นข้อดีของ stack (เขียนกลับกัน)

A5. ตอบ d) memory traffic สูง และ accumulator เป็น bottleneck

มี register เดียว จึงต้องเก็บค่ากลางลง memory บ่อย

A6. ตอบ b) poor orthogonality (operand สองตัวมีสิทธิ์ไม่เท่ากัน) และ CPI แปรผัน

operand ตัวหนึ่งอยู่ใน memory ได้ แต่อีกตัวต้องเป็น register

A7. ตอบ b) มี special case น้อย ใช้ทุก addressing mode กับทุก data type/instruction ได้

เป็นคุณสมบัติที่ช่วย compiler writer

A8. ตอบ a) x86 แปลคำสั่ง CISC เป็นคำสั่งภายในคล้าย RISC (micro-ops) ก่อนทำ parallelism

เส้นแบ่งระหว่าง CISC กับ RISC จึงไม่ชัดแล้ว

A9. ตอบ b) Acorn RISC Machine

เด่นเรื่องประหยัดพลังงาน

A10. ตอบ b) decode ง่ายและทำ pipeline ได้ง่าย

แลกกับ code ที่ใหญ่ขึ้น

A11. ตอบ b) MIPS เร็วกว่าประมาณ 3 เท่า

$T_{MIPS}/T_{VAX} = 2 \times 1/6 = 1/3$

A12. ตอบ d) `XOR R1,R1,R1`

ไม่ต้องใช้ immediate encode สั้น และ HW รู้จัก idiom นี้

A13. ตอบ b) Code motion

ย้ายโค้ด loop-invariant ออกนอก loop (global optimization)

A14. ตอบ b) Strength reduction

ใช้ operation ที่ถูกกว่า (machine-dependent)

A15. ตอบ a) ลำดับคำสั่งที่ไม่มี branch เข้าหรือออกตรงกลาง

local optimization ทำภายใน basic block และ basic block ใหญ่ทำให้มี ILP มาก (Ch7)

A16. ตอบ b) Alias (pointer)

compiler ไม่รู้ว่า pointer จะแก้ตัวแปรไหน จึงต้องอ่านเขียน memory จริง

A17. 11, 10, 4, 10, 8

Mem-Mem แบบ 2-operand ใช้ 7 คำสั่ง

A18. B = B+C+C, D = (B+C) – (B+C+C) = –C → บางคำสั่งงานเพิ่ม บางคำสั่งงานลด

optimization ไม่ได้ลดงานเสมอ compiler ต้องชั่งน้ำหนัก ("locally simple, globally complex")

A19. ย้าย `hs=3600; ds=86400;` ออกนอก loop (code motion + constant folding)

ใน loop เหลือแค่ `h=a[x]/hs; d=a[x]/ds;`

Ch3 (2): Endian, Alignment, Addressing, Control Flow, Exceptions

สรุปก่อนทำโจทย์

- **Little endian:** LSB อยู่ address ต่ำ (Intel) · **Big endian:** MSB อยู่ address ต่ำ (PowerPC, Sun, Photoshop) · ARM เลือกได้
- **Endian performance:** บวกเลข → little ดีกว่า (ได้ byte ต่ำก่อน) · เปรียบเทียบ → big ดีกว่า (ได้ sign bit ก่อน)
- **Restricted alignment:** address mod size = 0 ภาระอยู่ที่ compiler (padding) HW trap และไม่เสียเวลา → ชนะ · **Unrestricted:** HW ต้องแก้เอง อาจต้องเข้าถึง 2 ครั้ง

Mode	ตัวอย่าง	C
Immediate	`LDI r1, #4`	A = 4
Register	`ADD r1,r2,r3`	A = B + C
Displacement	`LD r1, 100(r2)`	A = G.age / local var
Register indirect	`LD r1, (r2)`	A = `*p`
Index	`LD r1, (r2+r3)`	A = D[i]
Direct	`LD r1, 1000`	global
Memory indirect	`LD r1, @(r2)`	A = `**p`
Auto-inc/dec	`LD r1, (r2)+`	D[i++] / D[i−−]
Scale	`LD r1, (r2)[r3*4]`	E[i]

- **PC-relative** → relocatable · shared library ใช้ register (`CALL $r14`)
- **Caller-save** (ผู้เรียก save) vs **callee-save** (subroutine save เฉพาะที่ใช้ → คุ้มกว่า) · x86 PUSHA/POPA
- **Exception** = unprogrammed control flow: **interrupt** (HW ภายนอก) / **trap** (software) → save status + PC → handler → restore
- **Vector table:** mem[Base + n × address size] · **Handler table:** Base + n × handler size · **Fixed entry:** กระโดดไป 0x1000 แล้ว software ดู source register

โจทย์ (20 ข้อ)

ก. Endian และ Alignment

D1. เก็บ 0x11223344 แบบ big endian ที่ 0x1000 ไบต์ที่ 0x1003 คือ

a) 0x11 b) 0x22

c) 0x33 d) 0x44

D2. เก็บ 0xDEADBEEF แบบ little endian ที่ 0x40 ไบต์ที่ 0x40 คือ

a) 0xDE
b) 0xAD
c) 0xBE
d) 0xEF

D3. สำหรับการเปรียบเทียบเลข 32 บิตบน bus 8 บิต endian แบบใดเร็วกว่า

a) Little: ได้ byte ต่ำก่อน
b) Big: ได้ byte ที่มี sign bit ก่อน
c) Little: carry ถูกต้อง
d) เท่ากัน

D4. ภายใต้ restricted alignment address ใดเก็บ int (4 bytes) ได้

a) 0x1002
b) 0x1006
c) 0x100C
d) 0x1011

D5. ภายใต้ restricted alignment `char a; double b;` เริ่มที่ 0 ตัวแปร b อยู่ที่ address ใด

a) 1
b) 2
c) 4
d) 8

D6. ข้อใดถูกเกี่ยวกับ unrestricted alignment

a) compiler ต้องใส่ padding
b) hardware trap ทุกครั้งที่ไม่ aligned
c) hardware ต้องตรวจและแก้เอง ซึ่งอาจต้องเข้าถึง memory 2 ครั้งขึ้นไป
d) เร็วกว่า restricted เสมอ

ข. Addressing Modes และ Control Flow

D7. `LD r1, 100(r2)` ที่ใช้อ่าน field ของ struct (`A = G.age`) คือ addressing mode ใด

a) Register indirect
b) Displacement
c) Memory indirect
d) Scale

D8. โค้ด C `A = **p;` ตรงกับ addressing mode ใด

a) Register indirect
b) Memory indirect
c) Displacement
d) Auto-increment

D9. `LD r1, (r2)+` ตรงกับโค้ด C แบบใด

a) A = `*p`
b) A = D[i++]
c) A = `**p`
d) A = D[i−−]

D10. ข้อดีหลักของ branch แบบ PC-relative คือ

a) กระโดดไปทุก address ได้
b) ไม่ต้องใช้ adder
c) โปรแกรม relocatable
d) memory เร็วขึ้น

D11. Callee-save หมายถึงข้อใด

a) ผู้เรียก PUSH ทุก register ก่อน CALL
b) ไม่มีใคร save
c) OS เป็นผู้ save
d) subroutine save เฉพาะ register ที่ตัวเองจะแก้ แล้ว restore ก่อน RET

ค. Exceptions

D12. Exception ที่เกิดจาก software เช่น system call หรือหารด้วยศูนย์ เรียกว่า

a) Interrupt
b) Trap
c) Vector
d) Bubble

D13. การหา handler แบบ Fixed Entry ทำงานอย่างไร

a) อ่าน address จากตารางตามหมายเลข
b) กระโดดไปที่ Base + n × handler size
c) กระโดดไป address คงที่ แล้ว software อ่าน interrupt-source register เพื่อตัดสินใจ
d) hardware เลือกตาม priority

D14. Vector table ที่ base 0x8000 และ address 4 bytes เมื่อเกิด interrupt หมายเลข 5 CPU อ่าน address ของ handler จาก

a) 0x8005
b) 0x8014
c) 0x8050
d) 0x8020

D15. Handler table ที่ base 0x4000 และ handler ขนาด 32 bytes เมื่อเกิด interrupt หมายเลข 3 CPU กระโดดไปที่

a) 0x400C
b) 0x4003
c) 0x4060
d) 0x4096

ง. ตอบสั้น

D16. เก็บ 0x0A0B0C0D ที่ 0x100 แบบ little endian จงเขียน byte ที่ 0x100–0x103 และบอกว่าถ้าเป็น big endian byte ที่ 0x102 คืออะไร

D17. Restricted alignment: `char a; int b; short c; double d; char e;` เริ่มที่ 0 ตัวแปร d เริ่มที่ address ใด และไบต์สุดท้ายที่ใช้คือ address ใด

D18. (Exercise ในสไลด์) วาง `char a; short b; int c; char d; long e; double f;` แบบ restricted alignment รวมใช้กี่ bytes และถ้าเรียงใหม่จะเหลือเท่าใด

D19. Interrupt หมายเลข 6 (ก) vector table base 0x2000 address 32 bits อ่านจาก address ใด (ข) handler table base 0x2000 handler 64 bytes กระโดดไปที่ใด

D20. เมื่อเกิด exception CPU ต้องทำอะไรตามลำดับ (ตอบเป็นขั้นตอน)

เฉลย: Ch3 (2): Endian, Alignment, Addressing, Control Flow, Exceptions

ข้อ	D1	D2	D3	D4	D5	D6	D7	D8	D9	D10
ตอบ	d	d	b	c	d	c	b	b	b	c

ข้อ	D11	D12	D13	D14	D15
ตอบ	d	b	c	b	c

D1. ตอบ d) 0x44

MSB อยู่ address ต่ำ: 11, 22, 33, 44 (ตัวลวง a เป็นคำตอบแบบ little endian)

D2. ตอบ d) 0xEF

LSB อยู่ address ต่ำ

D3. ตอบ b) Big: ได้ byte ที่มี sign bit ก่อน

little endian เหมาะกับการบวก

D4. ตอบ c) 0x100C

0x100C mod 4 = 0

D5. ตอบ d) 8

double ต้องหาร 8 ลงตัว (padding 1–7)

D6. ตอบ c) hardware ต้องตรวจและแก้เอง ซึ่งอาจต้องเข้าถึง memory 2 ครั้งขึ้นไป

a และ b เป็นลักษณะของ restricted alignment

D7. ตอบ b) Displacement

M[R + ค่าคงที่]

D8. ตอบ b) Memory indirect

M[M[R2]] (register indirect คือ `*p`)

D9. ตอบ b) A = D[i++]

auto-increment: อ่าน M[R2] แล้วเพิ่ม R2

D10. ตอบ c) โปรแกรม relocatable

ปลายทางไม่ผูกกับ address จริง และ branch ส่วนใหญ่กระโดดใกล้ ๆ

D11. ตอบ d) subroutine save เฉพาะ register ที่ตัวเองจะแก้ แล้ว restore ก่อน RET

caller ไม่รู้ว่า callee ใช้ register ไหน จึงอาจ save เกินจำเป็น

D12. ตอบ b) Trap

interrupt มาจาก hardware ภายนอก

D13. ตอบ c) กระโดดไป address คงที่ แล้ว software อ่าน interrupt-source register เพื่อตัดสินใจ

a = vector table และ b = handler table

D14. ตอบ b) 0x8014

0x8000 + 5 × 4 = 0x8000 + 0x14 (0x8050 เป็นคำตอบของ handler table ขนาด 16 B)

D15. ตอบ c) 0x4060

3 × 32 = 96 = 0x60 (ตัวลวง d ใช้ 96 เป็น hex)

D16. little: 0D 0C 0B 0A · big ที่ 0x102 = 0x0C

big endian: 0A 0B 0C 0D

D17. d เริ่มที่ 16, ไบต์สุดท้ายคือ 24

a @0, b @4–7, c @8–9, d @16–23 (pad 10–15), e @24 → ใช้ 25 bytes

D18. 32 bytes · เรียงจากใหญ่ไปเล็กเหลือ 24 bytes

a @0, b @2–3, c @4–7, d @8, pad 9–15, e @16–23, f @24–31 · เรียงใหม่ได้ 8 + 8 + 4 + 2 + 1 + 1 = 24

D19. (ก) 0x2018 (ข) 0x2180

6 × 4 = 24 = 0x18 และ 6 × 64 = 384 = 0x180

D20. save status → save PC → เรียก handler → ทำงาน → restore status → restore PC (เช่น IRET)

กลไกคล้าย subroutine call แต่เกิดโดยที่โปรแกรมไม่ได้สั่ง

Ch4: Single Cycle Processor (nanoLADA)

สรุปก่อนทำโจทย์

- **Design steps:** (1) analyze ISA → RTL (2) select components (3) assemble datapath (4) analyze control points (5) assemble control logic
- **Format:** R = op 6 · rs 5 · rt 5 · rd 5 · reserved 11 · I = op 6 · rs 5 · rt 5 · **imm 16** · J = op 6 · **addr 26**
- **RTL:** ORI R[rt] ← R[rs] OR zero_ext(imm) · ORUI ... OR zero_pad(imm) · ADD R[rd] ← R[rs] + R[rt] · LW R[rt] ← MEM[R[rs] + sign_ext(imm)] · SW MEM[...] ← R[rt] · BEQ PC ← PC + 4 + sign_ext(imm) × 4 · JMP PC ← addr × 4
- **Extender:** sign extend (LW/SW/BEQ) · zero extend 0x0000_imm (ORI) · zero pad imm_0000 (ORUI)
- **Register file:** 32 × 32 bits, อ่าน 2 port, เขียน 1 port · PC + 4 ใช้ adder 30 bits ได้ (2 บิตล่างเป็น 0)

Instr	sel_pc	sel_addpc	sel_wr	sel_b	sel_data	reg_wr	mem_wr	alu_ops
ORI/ORUI	0	0	1 (rt)	1 (imm)	0	1	0	01
ADD	0	0	0 (rd)	0 (reg)	0	1	0	00
LW	0	0	1	1	1 (mem)	1	0	00
SW	0	0	x	1	x	0	1	00
BEQ	0	zero	x	0	x	0	0	10
JMP	1	x	x	x	x	0	0	xx

- **Critical path (LW):** PC 10 + IMem 40 + Control 40 + Reg 20 + MUX 10 + ALU 60 + DMem 40 + MUX 10 = **230 ns** → CPI = 1 แต่ทุกคำสั่งต้องใช้ 230 ns

โจทย์ (17 ข้อ)

ก. ISA และ Components

S1. ขั้นตอนแรกของการออกแบบ processor ตามสไลด์คือ

a) ประกอบ control logic
b) เลือก components
c) วิเคราะห์ ISA โดยเขียนแต่ละคำสั่งเป็น RTL
d) หา critical path

S2. ใน nanoLADA คำสั่ง I-type มี immediate กี่บิต และ J-type มี address กี่บิต

a) 16 และ 26
b) 16 และ 32
c) 11 และ 26
d) 5 และ 16

S3. Register file ของ nanoLADA มีกี่ port

a) อ่าน 1, เขียน 1
b) อ่าน 2, เขียน 2
c) อ่าน 2, เขียน 1
d) อ่าน 3, เขียน 1

S4. คำสั่งใดใช้ extender แบบ zero extend

a) ORI
b) ORUI
c) LW
d) BEQ

S5. หลังรัน `ORUI r1, r0, #0xBEEF` ค่าของ r1 คือ

a) 0x0000BEEF
b) 0xFFFFBEEF
c) 0xBEEF0000
d) 0xBEEFBEEF

S6. ทำไม adder สำหรับ PC + 4 ใช้แค่ 30 bits ก็พอ

a) เพราะ PC มี 30 bits
b) เพราะ 2 บิตล่างของ PC เป็น 0 เสมอ (word aligned) จึงบวก 1 ที่ PC[31:2]
c) เพราะ memory มีขนาด 1 GB
d) เพราะ JMP ใช้ 26 bits

ข. Control Signals

S7. ค่า control ของคำสั่ง ADD ข้อใดถูก

a) sel_wr = 1 (rt), sel_b = 1 (imm)
b) sel_wr = 0 (rd), sel_b = 0 (R[rt])
c) sel_data = 1 (memory)
d) mem_wr = 1

S8. ค่า control ของคำสั่ง LW ข้อใดถูก

a) sel_b = 1, sel_data = 1, reg_wr = 1
b) sel_b = 0, sel_data = 0, reg_wr = 1
c) sel_b = 1, sel_data = x, mem_wr = 1
d) sel_pc = 1, reg_wr = 0

S9. คำสั่งใดมี reg_wr = 0 และ mem_wr = 1

a) LW
b) SW
c) BEQ
d) ORI

S10. ค่า control ของคำสั่ง BEQ ข้อใดถูก

a) alu_ops = 10 (SUB), sel_addpc = zero, reg_wr = 0
b) alu_ops = 00, reg_wr = 1
c) sel_pc = 1, mem_wr = 1
d) sel_data = 1, reg_wr = 1

S11. `BEQ r1, r2, imm` อยู่ที่ PC = 0x0040, imm = −3 และ R[r1] = R[r2] ค่า PC ถัดไปคือ

a) 0x0034
b) 0x0038
c) 0x003D
d) 0x0044

S12. `JMP` มี address field = 0x40 ค่า PC ถัดไปคือ

a) 0x40
b) 0x44
c) 0x100
d) 0x104

ค. Critical Path

S13. คำสั่งใดกำหนด critical path ของ single-cycle nanoLADA และ clock สูงสุดประมาณเท่าใด

a) ADD, 5.3 MHz
b) LW, 4.35 MHz
c) BEQ, 230 MHz
d) JMP, 43.5 MHz

ง. ตอบสั้น

S14. ด้วย delay ตามสไลด์ คำสั่ง ADD ใช้เวลาจริงเท่าใด และ clock ของ single cycle ต้องตั้งเป็นเท่าใด

S15. (Exercise ในสไลด์) benchmark 2 ล้านคำสั่ง บน single cycle ที่มี critical path 230 ns ใช้ CPU time เท่าใด

S16. เขียน control signals ของ ORI ทั้งแถว (sel_pc, sel_addpc, sel_wr, sel_b, sel_data, reg_wr, mem_wr, alu_ops)

S17. จะสร้างค่าคงที่ 0x12345678 ใน r1 ด้วยคำสั่ง nanoLADA 2 คำสั่งได้อย่างไร

เฉลย: Ch4: Single Cycle Processor (nanoLADA)

ข้อ	S1	S2	S3	S4	S5	S6	S7	S8	S9	S10
ตอบ	c	a	c	a	c	b	b	a	b	a

ข้อ	S11	S12	S13
ตอบ	b	c	b

S1. ตอบ c) วิเคราะห์ ISA โดยเขียนแต่ละคำสั่งเป็น RTL

ต้องรู้ว่าแต่ละคำสั่งทำอะไรก่อน จึงจะรู้ว่า datapath ต้องมีอะไร

S2. ตอบ a) 16 และ 26

I = 6 + 5 + 5 + 16 และ J = 6 + 26

S3. ตอบ c) อ่าน 2, เขียน 1

มี write port เดียว จึงเกิด structural hazard ใน pipeline ได้ (Ch6)

S4. ตอบ a) ORI

ORUI ใช้ zero pad และ LW/BEQ ใช้ sign extend

S5. ตอบ c) 0xBEEF0000

zero pad ให้ imm ไปอยู่ 16 บิตบน (a = zero extend และ b = sign extend)

S6. ตอบ b) เพราะ 2 บิตล่างของ PC เป็น 0 เสมอ (word aligned) จึงบวก 1 ที่ PC[31:2]

PC + 4 = {PC[31:2] + 1, 2'b00}

S7. ตอบ b) sel_wr = 0 (rd), sel_b = 0 (R[rt])

R-type เขียนลง rd และใช้ register เป็น operand B

S8. ตอบ a) sel_b = 1, sel_data = 1, reg_wr = 1

b = ADD, c = SW, d = JMP

S9. ตอบ b) SW

SW เป็นคำสั่งเดียวที่เขียน memory

S10. ตอบ a) alu_ops = 10 (SUB), sel_addpc = zero, reg_wr = 0

ใช้ SUB แล้วดู zero flag เพื่อเลือก PC

S11. ตอบ b) 0x0038

0x44 + (−3 × 4) = 0x44 − 0xC = 0x38 (a ลืมบวก 4 และ c ลืมคูณ 4)

S12. ตอบ c) 0x100

PC ← addr × 4 (JMP ไม่ได้เป็น PC-relative)

S13. ตอบ b) LW, 4.35 MHz

LW ใช้ทุกส่วนรวม 230 ns → 1/230 ns ≈ 4.35 MHz

S14. ADD = 190 ns, clock = 230 ns

10 + 40 + 40 + 20 + 10 + 60 + 10 = 190 (ไม่ผ่าน data memory) แต่ clock ต้องรอ LW → เสียเวลาเปล่า 40 ns ต่อ ADD

S15. 0.46 s

$2 \times 10^6 \times 1 \times 230$ ns

S16. 0, 0, 1, 1, 0, 1, 0, 01

เขียนลง rt (sel_wr = 1), ใช้ imm (sel_b = 1), เขียนผล ALU (sel_data = 0), alu_ops = OR

S17. `ORUI r1, r0, #0x1234` แล้ว `ORI r1, r1, #0x5678`

ORUI ใส่ 16 บิตบน (0x12340000) แล้ว ORI ใส่ 16 บิตล่าง

Ch5: Multiple Cycle Processor

สรุปก่อนทำโจทย์

- **ปัญหาของ single cycle:** cycle ยาว ทุกคำสั่งต้องรอเท่าคำสั่งที่ช้าที่สุด
- **วิธีแก้:** แบ่ง datapath แล้วใส่ intermediate registers (IR, REG_A, REG_B, REG_S, REG_M, REG_WR) ให้แต่ละส่วน**ใช้เวลาเท่า ๆ กัน** และคำสั่งง่ายข้ามขั้นที่ไม่ใช้ได้ ใช้ hardware ร่วมกันได้ (ALU คำนวณ PC + 4, รวม memory)
- **5 ขั้น:** Fetch (IR ← MEM[PC]; PC ← PC + 4) → Decode → Execute → Memory → Write back
- **จำนวน cycle: LW 5 · ORI/ORUI/ADD/SW 4 · BEQ/JMP 3** (BEQ บวกแค่ imm × 4 เพราะ PC + 4 ไปแล้วตอน fetch)
- **SW:** 1 Fetch · 2 A ← R[rs], B ← sign_ext, **M ← R[rt]** · 3 S ← A + B · 4 **MEM[S] ← M**
- **Control = FSM** (Fetch → Decode → ALU_Exec/ADD_Calc/BEQ/JMP → ...) implement ด้วย sequential circuit หรือ **microprogram** (เปลี่ยน Map ROM แล้วทำ ISA อื่นได้)
- **Multi-cycle ไม่ได้เร็วกว่าเสมอ:** เทียบ $CPI_{avg} \times T_{c,multi}$ กับ $T_{c,single}$

โจทย์ (17 ข้อ)

ก. แนวคิด

M1. ปัญหาหลักของ single-cycle processor คือ

a) CPI สูง
b) ทุกคำสั่งต้องใช้เวลาเท่าคำสั่งที่ช้าที่สุด (cycle ยาว)
c) มี hazard
d) ใช้ register มากเกินไป

M2. วิธีลด cycle time ของ multi-cycle ตามสไลด์คือ

a) เพิ่ม clock โดยไม่แก้ datapath
b) แบ่ง datapath แล้วใส่ intermediate register ให้แต่ละส่วนใช้เวลาใกล้เคียงกัน
c) ลดจำนวนคำสั่ง
d) ใช้ cache ใหญ่ขึ้น

M3. ใน multi-cycle nanoLADA คำสั่ง LW, SW และ JMP ใช้กี่ cycle ตามลำดับ

a) 5, 5, 3
b) 5, 4, 3
c) 4, 4, 3
d) 5, 4, 2

M4. cycle ที่ 4 ของคำสั่ง SW ทำอะไร

a) IR ← MEM[PC]
b) REG_S ← REG_A + REG_B
c) MEM[REG_S] ← REG_M
d) R[rt] ← REG_WR

M5. ทำไม cycle ที่ 3 ของ BEQ ใน multi-cycle บวก PC ด้วย imm × 4 อย่างเดียว ไม่บวก 4 เพิ่ม

a) เพราะ BEQ ไม่ใช้ PC
b) เพราะ PC ถูกบวก 4 ไปแล้วใน cycle Fetch
c) เพราะ imm รวม 4 ไว้แล้ว
d) เพราะ ALU บวกไม่ได้

M6. Control unit ของ multi-cycle processor มีลักษณะแบบใด

a) Combinational logic ล้วน
b) Finite state machine
c) ROM ที่ไม่มี state
d) Pipeline

M7. ข้อดีสำคัญของการ implement control ด้วย microprogram คือ

a) เร็วกว่า hardwired เสมอ
b) เปลี่ยน microprogram (Map ROM) แล้วทำ ISA อื่นบน hardware เดิมได้
c) ไม่ต้องมี state
d) ลด IC

M8. ทำไมไม่ให้ programmer เขียนโปรแกรมด้วย microinstruction โดยตรง

a) microinstruction ช้ากว่า
b) ไม่ Turing complete
c) ต้องใช้ OS พิเศษ
d) microinstruction ผูกกับ organization จึงไม่ compatible ข้ามรุ่น

ข. คำนวณ

M9. Single cycle มี cycle time 240 ns ส่วน multi-cycle 60 ns ถ้าต้องการให้ multi-cycle เร็วอย่างน้อยเท่ากัน CPI ต้องเป็นเท่าใด

a) ≥ 4
b) ≤ 4
c) ≤ 5
d) $= 1$

M10. multi-cycle (20 ns) รัน LW 3, ADD 2, BEQ 1 คำสั่ง ใช้เวลาเท่าใด

a) 120 ns
b) 480 ns
c) 520 ns
d) 600 ns

M11. multi-cycle (25 ns) รัน ADD 4, SW 2, BEQ 2, JMP 1 คำสั่ง ใช้เวลาเท่าใด

a) 225 ns
b) 825 ns
c) 900 ns
d) 1125 ns

ค. ตอบสั้น

M12. (Exercise ในสไลด์) single cycle 210 ns และ multi-cycle 45 ns CPI เฉลี่ยต้องไม่เกินเท่าใดจึงจะได้ performance อย่างน้อยเท่าเดิม

M13. (Exercise ในสไลด์) cycle time 40 ns โค้ด: `LW; LW; BEQ (not taken); ADD; SW` ใช้เวลาเท่าใด

M14. Single 200 ns, multi 50 ns และ mix: LW 20%, SW 10%, R-type/ORI 50%, BEQ 20% จงหา CPI และบอกว่าเครื่องใดเร็วกว่า

M15. Single 250 ns, multi 60 ns และ mix: LW 25%, SW 10%, R/ORI 45%, BEQ 15%, JMP 5% จงหา CPI และบอกว่าเครื่องใดเร็วกว่า

M16. (ตัวอย่างในสไลด์) 100 คำสั่ง: single cycle 45 ns/cycle และ multi-cycle 10 ns/cycle CPI 4.6 แบบใดเร็วกว่า

M17. จะดัดแปลง datapath ให้ ALU ใช้คำนวณ PC + 4 แทน adder แยกได้อย่างไร

เฉลย: Ch5: Multiple Cycle Processor

ข้อ	M1	M2	M3	M4	M5	M6	M7	M8	M9	M10
ตอบ	b	b	b	c	b	b	b	d	b	c

ข้อ	M11
ตอบ	b

M1. ตอบ b) ทุกคำสั่งต้องใช้เวลาเท่าคำสั่งที่ช้าที่สุด (cycle ยาว)

CPI = 1 อยู่แล้ว แต่ cycle time ยาวเพราะ critical path

M2. ตอบ b) แบ่ง datapath แล้วใส่ intermediate register ให้แต่ละส่วนใช้เวลาใกล้เคียงกัน

cycle time = เวลาของส่วนที่ยาวที่สุด จึงต้อง balance

M3. ตอบ b) 5, 4, 3

SW ไม่มี write back และ JMP ใช้ fetch + decode + เปลี่ยน PC

M4. ตอบ c) MEM[REG_S] ← REG_M

d คือ write back ซึ่ง SW ไม่มี

M5. ตอบ b) เพราะ PC ถูกบวก 4 ไปแล้วใน cycle Fetch

Fetch ทำ PC ← PC + 4 ทุกคำสั่ง

M6. ตอบ b) Finite state machine

แต่ละ state = 1 cycle และกำหนด control signals ของ cycle นั้น

M7. ตอบ b) เปลี่ยน microprogram (Map ROM) แล้วทำ ISA อื่นบน hardware เดิมได้

และทำ control ของ CISC ที่ซับซ้อนได้ง่าย

M8. ตอบ d) microinstruction ผูกกับ organization จึงไม่ compatible ข้ามรุ่น

ISA เป็น abstraction ที่ทำให้ software portable

M9. ตอบ b) ≤ 4

CPI × 60 ≤ 240

M10. ตอบ c) 520 ns

15 + 8 + 3 = 26 cycles × 20 ns

M11. ตอบ b) 825 ns

16 + 8 + 6 + 3 = 33 cycles × 25 ns

M12. CPI ≤ 4.67

210 / 45 = 4.67

M13. 21 cycles = 840 ns

5 + 5 + 3 + 4 + 4 = 21

M14. CPI = 4.0, เร็วเท่ากัน (200 ns ต่อคำสั่ง)

0.2(5) + 0.1(4) + 0.5(4) + 0.2(3) = 4.0 → 4.0 × 50 = 200 ns

M15. CPI = 4.05, multi เร็วกว่าเล็กน้อย (243 vs 250 ns)

1.25 + 0.40 + 1.80 + 0.45 + 0.15 = 4.05 → 4.05 × 60 = 243 ns · speedup ≈ 1.03

M16. single 4500 ns, multi 4600 ns → single เร็วกว่า

แสดงว่า multi-cycle ไม่ได้เร็วกว่าเสมอ (pipeline ในตัวอย่างเดียวกันใช้ 1030 ns)

M17. ใส่ MUX ที่ input A (เลือก PC หรือ REG_A) และ input B (เลือก 4, REG_B หรือ imm × 4) แล้วใช้ ALU ใน cycle Fetch

ทำได้เพราะ Fetch ไม่ได้ใช้ ALU ทำอย่างอื่น

Ch6: Pipelining และ Hazards

สรุปก่อนทำโจทย์

- **Pipelining** เพิ่ม **throughput** ไม่ลด latency · ความเร็วถูกจำกัดด้วย **stage ที่ช้าที่สุด** · stage ไม่สมดุลและเวลา fill/drain ลด speedup
- **$T = (k + n - 1) \times T_c$** → speedup ≈ k · laundry 4 ขั้น × 30 นาที: k loads = (k + 3)/2 ชั่วโมง
- **5 stages:** IF → ID (Reg/Dec) → EX → MEM → WB · register file เขียนครึ่งแรกอ่านครึ่งหลังของ cycle
- **Structural** (resource ไม่พอ: memory เดียว, WB พร้อมกัน) → เพิ่ม resource (แยก I/D cache), stall/bubble หรือ redesign (ให้ R-type ใช้ 5 stages)
- **Data:** RAW = Rregs(i) ∩ Wregs(j) (forwarding แก้ได้เกือบหมด) · WAW = Wregs(i) ∩ Wregs(j) · WAR = Wregs(i) ∩ Rregs(j) · ไม่มี RAR · in-order 5-stage มีปัญหาจริงแค่ RAW
- **Load-use:** ต้อง stall 1 cycle แม้มี forwarding (ถ้าไม่มี forwarding ต้องรอ 3) → compiler reschedule (delayed load) หรือแทรก NOP
- **Control:** stall (2 cycles), branch prediction (≥ 50%, history ~90%), **branch delay slot** (คำสั่งหลัง branch ทำเสมอ ต้องให้ compiler ช่วย)
- **Hazard detection:** เทียบ rs/rt กับ register ปลายทางที่รอเขียนใน EX/MEM/WB

โจทย์ (21 ข้อ)

ก. แนวคิด Pipeline

L1. Pipelining ปรับปรุงสิ่งใดเป็นหลัก

a) Latency ของแต่ละคำสั่ง
b) Instruction count
c) Throughput
d) Miss penalty

L2. 5 stages ของ pipeline ตามสไลด์เรียงอย่างไร

a) IF, EX, ID, MEM, WB
b) IF, ID, EX, MEM, WB
c) ID, IF, EX, WB, MEM
d) IF, ID, MEM, EX, WB

L3. ซักผ้าแบบ pipeline 4 ขั้นที่ใช้เวลา 30, 40, 30, 20 นาที เมื่อ pipeline เต็มแล้วผ้าจะเสร็จ 1 load ทุกกี่นาที

a) 20
b) 30
c) 40
d) 120

L4. Laundry 4 ขั้น ขั้นละ 30 นาที ทำแบบ pipeline 6 loads ใช้เวลาเท่าใด

a) 3 ชั่วโมง
b) 4.5 ชั่วโมง
c) 6 ชั่วโมง
d) 12 ชั่วโมง

ข. ชนิดของ Hazard

L5. การแยก instruction cache กับ data cache ช่วยแก้ hazard ชนิดใด

a) Structural
b) Data (RAW)
c) Control
d) WAW

L6. LW (5 stages) ตามด้วย ADD (4 stages ไม่มี MEM) แล้วเขียน register file ใน cycle เดียวกัน คือ hazard ชนิดใด

a) Structural
b) Data (RAW)
c) Control
d) WAR

L7. โค้ด `a = 7; b = a + 5;` เป็น dependency ชนิดใด

a) RAW
b) WAR
c) WAW
d) Structural

L8. โค้ด `a = sin(x); x = 10;` (sin ต้องเตรียมนาน) อาจเกิด hazard ชนิดใด

a) RAW
b) WAW
c) WAR
d) RAR

L9. ทำไมไม่มี hazard แบบ RAR (Read After Read)

a) hardware ห้ามอ่านซ้ำ
b) การอ่านไม่เปลี่ยนค่า ลำดับการอ่านจึงไม่มีผล
c) compiler ลบให้
d) เกิดแค่ใน VLIW

ค. วิธีแก้ Hazard

L10. 5-stage pipeline ที่มี full forwarding ถ้า LW ตามด้วยคำสั่งที่ใช้ค่านั้นทันที ต้อง stall กี่ cycle

a) 0
b) 1
c) 2
d) 3

L11. ลำดับคำสั่งใดยังต้อง stall แม้มี full forwarding

a) `ADD r1,..` ตามด้วย `SUB ..,r1,..`
b) `LW r1,0(r2)` ตามด้วย `ADD r3,r1,r4`
c) `SW r1,0(r2)` ตามด้วย `ADD r3,r4,r5`
d) `ADD r1,..` ตามด้วย `SW r1,0(r2)`

L12. วงจร hazard detection เปรียบเทียบอะไร

a) opcode ของคำสั่งที่ติดกัน
b) rs/rt ของคำสั่งปัจจุบัน กับ register ปลายทางที่รอเขียนใน EX/MEM/WB
c) PC ของทุก stage
d) address ของ memory

L13. ข้อใดอธิบาย branch delay slot ได้ถูกต้อง

a) hardware ยกเลิกคำสั่งหลัง branch ทุกครั้ง
b) คำสั่งถัดจาก branch ทำงานเสมอไม่ว่า branch จะ taken หรือไม่ และต้องให้ compiler หาคำสั่งมาใส่
c) เป็นการทาย branch ด้วย history
d) ไม่ต้องอาศัย compiler

L14. ตามสไลด์ branch prediction แบบใช้ branch history ทายถูกประมาณเท่าใด

a) 50%　　b) 75%

c) 90%　　d) 100%

ง. ตอบสั้น

L15. 5-stage pipeline (full forwarding) cycle time 2 ns รัน 200 คำสั่ง โดย 10% เป็น load-use และไม่มี stall อื่น ใช้เวลาเท่าใด

L16. โปรแกรม 7 คำสั่งของ nanoLADA (LW, LW, ADD, ORI, SW, BEQ, JMP) จงหาเวลาบน single cycle 230 ns, multi-cycle 50 ns และ ideal pipeline 5 stages 50 ns

---

L17. ระบุ dependency ทั้งหมดและจำนวน stall (full forwarding): `1: LW r1,0(r2)` · `2: ADD r3,r1,r1` · `3: SW r3,4(r2)` · `4: ADD r1,r4,r5`

L18. `1: LW r1,0(r2)` · `2: ADD r3,r1,r4` · `3: LW r6,4(r2)` · `4: SUB r7,r6,r3` · `5: ORI r8,r9,#5` (full forwarding) ใช้กี่ cycle และจะ reschedule ให้ไม่มี stall ได้อย่างไร

L19. `1: LW r2,0(r1)` · `2: ADD r3,r2,r4` · `3: SUB r2,r5,r6` · `4: SW r3,4(r1)` ทำไมย้าย SUB มาคั่นระหว่าง 1 กับ 2 เพื่อลด stall ไม่ได้ และต้องทำอย่างไร

L20. (Exercise ในสไลด์) `MOV R1,F0` (ช้า) · `LI F0,#20` · `LD R2,R1,0` · `ADDI R1,R1,#1` มี hazard อะไรบ้าง

L21. (Exercise ในสไลด์) reschedule loop นี้ให้ไม่มี stall โดยใช้ delayed load และ delayed branch: `LD F0,0(R1); ADD F4,F0,F2; SD 0(R1),F4; LD F1,8(R1); ADD F5,F1,F2; SD 8(R1),F5; SUB R1,R1,#16; BNEZ R1,Loop`

เฉลย: Ch6: Pipelining และ Hazards

ข้อ	L1	L2	L3	L4	L5	L6	L7	L8	L9	L10
ตอบ	c	b	c	b	a	a	a	c	b	b

ข้อ	L11	L12	L13	L14
ตอบ	b	b	b	c

L1. ตอบ c) Throughput

"doesn't help latency of single task, it helps throughput"

L2. ตอบ b) IF, ID, EX, MEM, WB

Fetch → Reg/Dec → Exec → Mem → Wr

L3. ตอบ c) 40

ถูกจำกัดด้วย stage ที่ช้าที่สุด (120 คือ latency ของ load เดียว)

L4. ตอบ b) 4.5 ชั่วโมง

2 + 5 × 0.5 = 4.5 h (12 h คือแบบ sequential)

L5. ตอบ a) Structural

memory เดียวใช้ทั้ง IF และ MEM พร้อมกันไม่ได้

L6. ตอบ a) Structural

register file มี write port เดียว (แก้ด้วย bubble หรือให้ R-type ใช้ 5 stages)

L7. ตอบ a) RAW

ตัวอย่างตรงจากสไลด์

L8. ตอบ c) WAR

คำสั่งหลังเขียน x ก่อนคำสั่งแรกจะอ่านเสร็จ

L9. ตอบ b) การอ่านไม่เปลี่ยนค่า ลำดับการอ่านจึงไม่มีผล

L10. ตอบ b) 1

ข้อมูลพร้อมตอนจบ MEM แต่ต้องใช้ตอนต้น EX ใน cycle เดียวกัน (3 คือกรณีไม่มี forwarding)

L11. ตอบ b) `LW r1,0(r2)` ตามด้วย `ADD r3,r1,r4`

load-use

L12. ตอบ b) rs/rt ของคำสั่งปัจจุบัน กับ register ปลายทางที่รอเขียนใน EX/MEM/WB

hazard = (rs = rw_ex AND regW_ex) OR ... ทั้ง rs และ rt

L13. ตอบ b) คำสั่งถัดจาก branch ทำงานเสมอไม่ว่า branch จะ taken หรือไม่ และต้องให้ compiler หาคำสั่งมาใส่

ถ้าหาคำสั่งอิสระไม่ได้ต้องใส่ NOP

L14. ตอบ c) 90%

ทายแบบคงที่ถูกอย่างน้อยประมาณ 50%

L15. 224 cycles = 448 ns

5 + 199 + 20 = 224 (CPI ที่มีผล ≈ 1.12)

L16. single 1610 ns, multi 1400 ns (28 cycles), pipeline 550 ns

pipeline = (5 + 7 − 1) × 50 → เร็วกว่า single 2.93 เท่า (ไม่ถึง 5 เพราะ n น้อย)

L17. RAW r1 (1→2), RAW r3 (2→3), WAR r1 (2→4), WAW r1 (1→4) · stall 1 cycle

เฉพาะ load-use (1→2) ที่ต้อง stall ส่วน WAR/WAW ไม่เป็นปัญหาใน in-order 5-stage → รวม 9 cycles

L18. 11 cycles → reschedule: LW r1; LW r6; ADD r3; ORI r8; SUB r7 = 9 cycles

- dependency: RAW r1 (1→2), RAW r6 (3→4), RAW r3 (2→4) ไม่มี WAR/WAW
- load-use 2 จุด จุดละ 1 stall → 9 + 2 = 11
- reschedule แล้วไม่สร้าง dependency ใหม่

L19. ละเมิด WAR (2→3) และ WAW (1→3) บน r2 → ต้อง rename ปลายทางของ SUB (เช่น r7) ก่อนจึงจะย้ายได้

ถ้าย้ายตรง ๆ ADD จะอ่าน r2 ของ SUB แทนค่าที่ load มา

L20. **WAR F0 (MOV/LI), RAW R1 (MOV→LD, MOV→ADDI), WAW R1 (MOV/ADDI), WAR R1 (LD/ADDI)**

MOV ใช้เวลานาน คำสั่งหลังจึงอาจเขียนก่อนที่ MOV จะอ่านหรือเขียนเสร็จ

L21. **LD F0; LD F1; ADD F4; ADD F5; SD 0(R1),F4; SUB R1,R1,#16; BNEZ; SD –8(R1),F5 (อยู่ใน delay slot)**

SD ตัวสุดท้ายย้ายมาหลัง SUB จึงต้องปรับ offset จาก 8 เป็น 8 – 16 = –8

Ch7: Instruction-Level Parallelism

สรุปก่อนทำโจทย์

- **Flynn:** SISD = uniprocessor · **SIMD** = vector/GPU (MMX, SSE, AVX, Altivec, CUDA) · **MISD** = fault-tolerant · **MIMD** = multicore/distributed
- **ILP** = จำนวนคำสั่งที่ทำพร้อมกันได้ · เทคนิค: vectorization, speculative execution, pipelining, superscalar (OoO, register renaming), VLIW

แบบ	ILP	T(n)	ลดอะไร
Pipeline	k	$(k + n - 1)\ T_c$	—
Superscalar (x)	kx	$(k + (n - x)/x)\ T_c$	**CPI** = 1/x (dynamic, HW)
Superpipeline (m)	mk	$(k + (n - 1)/m)\ T_c$	CPI = 1/m
VLIW (x)	kx	$(k + (n/x - x))\ T_c$	**IC** = n/x (static, compiler)

- **SuperSymmetry:** degree เท่ากันได้ performance เท่ากัน
- Superscalar อาจทำงาน **out-of-order** → เสี่ยง **WAR/WAW** → แก้ด้วย **register renaming** หรือ in-order completion
- **VLIW:** static scheduling → binary ไม่ compatible ข้ามรุ่น (Itanium, EOL 2019)
- **Dynamic vs Static:** dynamic ทำให้ legacy code เร็วขึ้นโดยไม่ต้อง recompile แต่ HW ซับซ้อนและใช้ power มาก ส่วน static ใช้ HW ง่ายแต่ compiler ทำยาก → **dynamic ชนะ**
- **Loop unrolling:** basic block ใหญ่ขึ้น (8 → 3.5 cycles/iter, speedup 2.29) แลกกับ code ที่ใหญ่ขึ้นและถูกจำกัดด้วยจำนวน register · **software pipelining** unroll เท่าที่จำเป็น
- **SMT** = ดึงคำสั่งจากหลาย thread · Intel เรียก **Hyper-Threading**

โจทย์ (15 ข้อ)

ก. แนวคิด

X1. ตาม Flynn's taxonomy ในสไลด์ MISD ใช้กับงานแบบใด

a) Uniprocessor
b) Vector processor
c) Fault-tolerant computing
d) Multicore

X2. ข้อใดเป็นตัวอย่างของ SIMD

a) Intel AVX
b) Server แบบ multicore
c) Uniprocessor 8088
d) ระบบที่ให้หลายเครื่องคำนวณแล้ว vote

X3. VLIW เพิ่ม performance โดยลดตัวแปรใดเป็นหลัก

a) IC
b) CPI
c) T_c
d) Miss rate

X4. ข้อใดถูกเกี่ยวกับ VLIW

a) ใช้ dynamic scheduling ใน hardware
b) ลด CPI แต่ไม่ลด IC
c) ใช้ static scheduling ทำให้ binary ข้ามรุ่น (Itanium → Itanium II) ไม่ compatible
d) Itanium ยังได้รับความนิยม

X5. Superscalar ที่ทำงานแบบ out-of-order เสี่ยงต่อ hazard ชนิดใดเพิ่มขึ้น

a) RAR
b) WAR และ WAW
c) Structural เท่านั้น
d) ไม่มี hazard

X6. เทคนิคใดกำจัด WAR และ WAW ได้

a) Forwarding
b) Branch prediction
c) Register renaming
d) Write buffer

X7. ข้อดีของ dynamic scheduling เทียบกับ static scheduling คือ

a) hardware ง่ายกว่า
b) compiler เห็น scope กว้างกว่า
c) ใช้ power ต่ำกว่า
d) software เดิมเร็วขึ้นบน hardware ใหม่โดยไม่ต้อง recompile

X8. ข้อใดไม่ใช่ข้อเสียหรือข้อจำกัดของ loop unrolling

a) code size ใหญ่ขึ้น
b) จำนวน register จำกัดจำนวนรอบที่ unroll ได้
c) ทำให้ basic block เล็กลง
d) ต้อง rename register

X9. Simultaneous Multithreading (SMT) ของ Intel มีชื่อว่า

a) Turbo Boost
b) Hyper-Threading
c) AVX
d) Itanium

ข. คำนวณ

X10. Superscalar degree 2, pipeline 5 stages รัน 1000 คำสั่ง (ideal) ใช้เวลากี่ T_c

a) 1004
b) 504
c) 502
d) 1000

X11. Superscalar degree 4, pipeline 5 stages รัน 1000 คำสั่ง ใช้เวลากี่ T_c

a) 254
b) 1004
c) 250
d) 259

X12. Superpipeline m = 3, k = 5 รัน n = 301 คำสั่ง ใช้เวลากี่ T_c

a) 105 b) 305

c) 101 d) 903

ค. ตอบสั้น

X13. (สไลด์) loop ปกติใช้ 8 cycles/iteration แต่หลัง unroll 4 รอบ + reschedule ใช้ 14 cycles ต่อ 4 iterations ได้ speedup เท่าใด และต้องทำอะไรเพิ่มเพื่อให้แต่ละ iteration อิสระกัน

X14. (สไลด์) loop รวม 19 cycles โดยงาน integer ใช้ 7 cycles และ FP ใช้ 12 cycles ถ้ามี 2 pipelines (Int + FP) ทำงานพร้อมกัน speedup สูงสุดเท่าใด

X15. เติมตาราง: superscalar degree x ลด ....., superpipeline m ลด ....., VLIW x ลด ..... และที่ degree เท่ากันทั้งสาม ได้ speedup .....

เฉลย: Ch7: Instruction-Level Parallelism

ข้อ	X1	X2	X3	X4	X5	X6	X7	X8	X9	X10
ตอบ	c	a	a	c	b	c	d	c	b	b

ข้อ	X11	X12
ตอบ	a	a

X1. ตอบ c) Fault-tolerant computing

SISD = uniprocessor, SIMD = vector, MIMD = multicore

X2. ตอบ a) Intel AVX

ทำ operation เดียวกับหลายข้อมูลในคำสั่งเดียว

X3. ตอบ a) IC

รวม x operations ใน 1 คำสั่ง (superscalar ลด CPI)

X4. ตอบ c) ใช้ static scheduling ทำให้ binary ข้ามรุ่น (Itanium → Itanium II) ไม่ compatible

Intel ประกาศ EOL ของ Itanium ในปี 2019

X5. ตอบ b) WAR และ WAW

คนที่มาทีหลัง (สั่งบะหมี่) อาจเสร็จก่อน (สั่งข้าว)

X6. ตอบ c) Register renaming

เป็น name dependence (forwarding แก้ RAW)

X7. ตอบ d) software เดิมเร็วขึ้นบน hardware ใหม่โดยไม่ต้อง recompile

a, b, c เป็นข้อดีของ static

X8. ตอบ c) ทำให้ basic block เล็กลง

loop unrolling ทำให้ basic block ใหญ่ขึ้น ซึ่งเป็นเป้าหมายของมัน

X9. ตอบ b) Hyper-Threading

ดึงคำสั่งจากหลาย thread เข้า pipeline เมื่อ thread เดียวเติม slot ไม่เต็ม

X10. ตอบ b) 504

5 + 998/2 = 504 (1004 คือ pipeline ธรรมดา)

X11. ตอบ a) 254

5 + 996/4 = 254 (250 ลืมเวลา fill)

X12. ตอบ a) 105

5 + 300/3 = 105

X13. 3.5 cycles/iter → speedup ≈ 2.29 · ต้อง register renaming

ถ้า unroll 5 รอบบน superscalar (Int + FP) จะได้ 12 cycles / 5 iter = 2.4 cycles/iter

X14. 19/12 ≈ 1.6

CPU time = max(Int, FP) = 12 cycles

X15. CPI (1/x), CPI (1/m), IC (n/x) · เท่ากัน (SuperSymmetry)

superscalar ทำ dynamic scheduling ใน HW ส่วน VLIW ทำ static scheduling ด้วย compiler

Ch8 (1): Memory Hierarchy และ Cache

สรุปก่อนทำโจทย์

- **Hierarchy:** register (SW จัดการ) → cache (HW) → memory → disk (HW + OS) · ระดับใกล้ CPU จะเล็ก เร็ว และแพง
- **DRAM** = capacitor, ถูก, ช้า, ต้อง refresh → main memory · **SRAM** = transistor, เร็ว, แพง → cache
- **Locality:** temporal (ใช้ซ้ำเร็ว ๆ นี้) · spatial (ใช้ข้อมูลที่อยู่ใกล้กัน)

```
offset = log₂(block B) · #sets = cache size / (block × ways) · index = log₂(#sets) · tag = ที่เหลือ
ฐานสิบ (direct): index = (addr mod cache size) div block · tag = addr div cache size
AMAT = Hit time + Miss rate × Miss penalty · 2 ระดับ: H₁ + MR₁ × (H₂ + MR₂ × MP₂)
CPI = CPI_ideal + Σ(accesses/instr × miss rate × penalty)
```

- **3C:** compulsory (ครั้งแรก) · capacity (cache เล็กเกิน, ping-pong) · conflict (ยังมีที่ว่างแต่ชนกัน → แก้ด้วย associativity) · fully associative ไม่มี conflict miss
- **Block ใหญ่:** spatial ดีขึ้น แต่ miss penalty สูงขึ้น และถ้าใหญ่เกินไป miss rate กลับเพิ่ม · **associativity สูง:** conflict ลดลง แต่ hit time เพิ่ม (n comparators)
- **Replacement:** LRU, round robin, FIFO, random · direct-mapped ไม่ต้องมี replacement policy
- **Write-back** (dirty bit, เร็ว, incoherent ใน multiprocessor) vs **write-through** (ช้า, เหมาะกับ multiprocessor) · **write buffer** saturate → แก้ด้วย **L2**
- **Software:** C เป็น row-major → วน r ข้างนอก c ข้างใน · array of struct ช่วยลด conflict miss

โจทย์ (23 ข้อ)

ก. แนวคิด

K1. ข้อใดถูกต้องเกี่ยวกับ DRAM และ SRAM

a) DRAM เร็วกว่า จึงใช้เป็น cache

b) SRAM เร็วและแพง จึงใช้เป็น cache ส่วน DRAM ถูกและหนาแน่น จึงใช้เป็น main memory

c) SRAM ต้อง refresh

d) DRAM ทำจาก flip-flop

K2. ข้อใดเป็นตัวอย่างของ temporal locality

a) อ่าน array เรียงไปเรื่อย ๆ

b) ตัวแปร i ใน loop ถูกใช้ซ้ำทุกรอบ

c) คำสั่งถูก fetch เรียงตาม address

d) block size 64 bytes

K3. "Ping-pong effect" (cache มี 1 line ทุก address ใหม่ miss หมด) เป็นกรณีสุดโต่งของ miss ชนิดใด

a) Compulsory

b) Capacity

c) Conflict

d) Coherence

K4. Fully associative cache ยังคงมี miss ชนิดใดบ้าง

a) Conflict อย่างเดียว
b) Compulsory และ Capacity
c) Compulsory, Capacity และ Conflict
d) ไม่มี miss เลย

K5. การเพิ่ม associativity จาก direct-mapped เป็น 4-way (ขนาดเท่าเดิม) มีผลข้อใดมากที่สุด

a) ลด compulsory miss
b) ลด conflict miss แต่ hit time อาจเพิ่มขึ้น
c) capacity miss เป็นศูนย์
d) ไม่ต้องมี tag

K6. ถ้า cache size คงที่แต่เพิ่ม block size มากเกินไป จะเกิดอะไรขึ้น

a) block น้อยลง temporal locality แย่ลง miss rate กลับเพิ่ม และ miss penalty สูงขึ้น
b) miss rate ลดลงเสมอ
c) hit time เป็นศูนย์
d) ไม่มี compulsory miss อีกต่อไป

K7. ทำไม direct-mapped cache จึงไม่ต้องมี replacement algorithm

a) เพราะไม่เคย miss
b) เพราะ block วางได้ที่เดียว จึงต้องแทนที่ตัวเดิมเสมอ
c) เพราะใช้ LRU อัตโนมัติ
d) เพราะ OS เป็นผู้เลือก

K8. Cache แบบ write-back ต้องมีบิตใดเพิ่มในแต่ละ line

a) Parity bit
b) Dirty bit
c) Sign bit
d) Carry bit

K9. นโยบายการเขียนแบบใดเหมาะกับระบบ multiprocessor มากกว่าตามสไลด์

a) Write-back เพราะเร็วกว่า
b) Write-through เพราะ memory เป็นปัจจุบันเสมอ
c) ไม่มีความต่าง
d) Write no-allocate เท่านั้น

K10. Write buffer saturate เพราะ memory ช้ามาก วิธีแก้ที่สไลด์เสนอคือ

a) เพิ่มขนาด write buffer ไปเรื่อย ๆ
b) เพิ่ม L2 cache
c) ปิด cache
d) ใช้ direct-mapped

K11. `int data[1000][1000]` ในภาษา C วนแบบใดเร็วกว่า

a) r ข้างนอก c ข้างใน (`data[r][c]`)
b) c ข้างนอก r ข้างใน
c) เร็วเท่ากัน
d) ขึ้นกับ OS

ข. Tag, Index, Offset

K12. Direct-mapped cache ขนาด 64 bytes, block 8 bytes, address 300 (ฐานสิบ) อยู่ที่ index ใด

a) 4
b) 5
c) 37
d) 44

K13. 2-way set associative ขนาด 8 KB, block 32 bytes มีกี่ sets

a) 64
b) 128
c) 256
d) 512

ค. ตอบสั้น

K14. (สไลด์) direct-mapped ที่มี Tag [31:10], Index [9:2], Byte offset [1:0] มี block size และ cache size เท่าใด และ address 0x00004098 อยู่ที่ index และ tag ใด

K15. Direct-mapped 4 KB, block 16 B, address 32 bits จงหา index และ tag ของ 0x00001234

K16. Direct-mapped 16 KB, block 32 B: (ก) tag/index/offset กี่บิต (ข) tag/index/offset ของ 0x00012345 (ค) 0x00016345 ชนกับ 0x00012345 ไหม

K17. 4-way set associative 64 KB, block 64 B, address 32 bits จงหาจำนวนบิตของ tag, index, offset

---

K18. 2-way set associative 32 KB, block 32 B จงหาจำนวนบิตของ tag, index, offset และจำนวน comparator

---

K19. CPI_{ideal} = 1, 30% เป็น load/store, I-miss 2%, D-miss 5%, penalty 100 cycles จงหา CPI จริง

K20. CPI_{ideal} = 1.2, 25% เป็น load/store, I-miss 1%, D-miss 4%, penalty 80 cycles จงหา CPI จริง

K21. L1 hit 1 cycle, miss 4% · L2 hit 12 cycles, local miss 25% · memory 200 cycles จงหา AMAT

K22. แบบเดิม hit 2 cycles, miss 8%, penalty 50 · แบบ 2-way hit 2.5 cycles, miss 6% แบบใดดีกว่า

K23. (สไลด์) memory op ใช้ 50 cycles, op อื่น 1 cycle และมี memory op 10% CPU รอ memory กี่เปอร์เซ็นต์ของเวลา

เฉลย: Ch8 (1): Memory Hierarchy และ Cache

ข้อ	K1	K2	K3	K4	K5	K6	K7	K8	K9	K10
ตอบ	b	b	b	b	b	a	b	b	b	b

ข้อ	K11	K12	K13
ตอบ	a	b	b

K1. ตอบ b) SRAM เร็วและแพง จึงใช้เป็น cache ส่วน DRAM ถูกและหนาแน่น จึงใช้เป็น main memory

DRAM ใช้ capacitor และต้อง refresh

K2. ตอบ b) ตัวแปร i ใน loop ถูกใช้ซ้ำทุกรอบ

a และ c เป็น spatial locality

K3. ตอบ b) Capacity

สไลด์เรียกว่า extreme capacity miss

K4. ตอบ b) Compulsory และ Capacity

วางได้ทุกที่จึงไม่มี conflict miss

K5. ตอบ b) ลด conflict miss แต่ hit time อาจเพิ่มขึ้น

ต้องมี 4 comparators และ MUX ใหญ่ขึ้น

K6. ตอบ a) block น้อยลง temporal locality แย่ลง miss rate กลับเพิ่ม และ miss penalty สูงขึ้น

optimal block size ต้องหาจาก benchmark

K7. ตอบ b) เพราะ block วางได้ที่เดียว จึงต้องแทนที่ตัวเดิมเสมอ

ไม่มีทางเลือกให้ตัดสินใจ

K8. ตอบ b) Dirty bit

บอกว่าต้องเขียนกลับ memory ตอน replace หรือไม่

K9. ตอบ b) Write-through เพราะ memory เป็นปัจจุบันเสมอ

write-back ทำให้ข้อมูล incoherent ในระบบ multiprocessor

K10. ตอบ b) เพิ่ม L2 cache

สไลด์บอกว่า buffer จะเต็ม "regardless the size"

K11. ตอบ a) r ข้างนอก c ข้างใน (`data[r][c]`)

C เก็บแบบ row-major จึงได้ spatial locality

K12. ตอบ b) 5

(300 mod 64) div 8 = 44 div 8 = 5 (tag = 4)

K13. ตอบ b) 128

8192 / (32 × 2) (256 คือจำนวน lines)

K14. block 4 B, cache 1 KB · index = 38, tag = 16

index = (0x4098 >> 2) AND 0xFF = 0x26 และ tag = 0x4098 >> 10 = 0x10

K15. index = 35, tag = 1 (offset = 4)

offset 4 bits, index 8 bits · 0x1234 = 0001 0010 0011 0100 → index = 0x23

K16. (ก) 18/9/5 (ข) tag 4, index 282, offset 5 (ค) ชน (conflict) เพราะ index เดียวกันแต่ tag = 5

0x16345 = 0x12345 + 16 KB (ขนาด cache พอดี) ถ้าเป็น 2-way ทั้งสองอยู่ด้วยกันได้

K17. tag 18, index 8, offset 6

#sets = 65536 / (64 × 4) = 256 (ถ้าลืมหาร ways จะได้ index 10 ซึ่งผิด)

K18. tag 18, index 9, offset 5, ใช้ 2 comparators

#sets = 32768 / 64 = 512

K19. 4.5

1 + 1 × 0.02 × 100 + 0.3 × 0.05 × 100 = 1 + 2 + 1.5 (ช้ากว่า perfect cache 4.5 เท่า)

K20. **2.8**

1.2 + 0.8 + 0.8

K21. **3.48 cycles**

MP_{L1} = 12 + 0.25 × 200 = 62 → 1 + 0.04 × 62 (ถ้าไม่มี L2 จะได้ 9 cycles)

K22. **เดิม 6 cycles, 2-way 5.5 cycles → 2-way ดีกว่า**

ต้องคำนวณ AMAT เพราะ associativity แลก hit time กับ miss rate

K23. **≈ 85%**

(0.1 × 50) / (0.9 + 5) = 5/5.9 ซึ่งเป็นแรงจูงใจให้ต้องมี cache

Ch8 (2): Virtual Memory และ TLB

สรุปก่อนทำโจทย์

- **ประโยชน์ 3 อย่าง:** (1) cost: รันโปรแกรม 4 GB บนเครื่อง 2 GB (ยืม disk, demand load, swap) (2) relocation: แต่ละ process มี address space ของตัวเอง (3) protection
- **Page** (virtual) / **page frame** (physical) · VA = **VPN** + offset · PA = **PPN** + offset (offset ไม่ถูกแปล)
- **Page table** map VPN → PPN มี valid bit และ permission · **มีต่อ process** · ขนาด = 2^(VPN bits) × PTE
- **Page fault** = miss ของ VM จัดการโดย **OS** (HW ไม่รู้จัก file system/swap) · miss penalty มหาศาล → page ใหญ่ (4 KB) และใช้ **write-back เท่านั้น**
- **TLB** = cache ของ page table entries · เล็ก (128–256 entries) จึงเป็น fully associative · TLB miss มีผลต่อ performance มาก
- **Overlapped cache & TLB:** ใช้ page offset เป็น index ของ cache พร้อมกับเปิด TLB → ต้องมี **cache size / ways ≤ page size**
- **Process switch:** flush TLB (หรือใช้ ASID) · page table ไม่ต้องลบ แค่เปลี่ยน page table register · cache แบบ physical ไม่ต้อง flush
- **Virtually addressed cache:** เร็วแต่มีปัญหา aliasing/shared memory

โจทย์ (14 ข้อ)

ก. แนวคิด

V1. ข้อใดไม่ใช่วัตถุประสงค์ของ virtual memory ตามสไลด์

a) รันโปรแกรมที่ใหญ่กว่า physical memory ได้
b) แต่ละ process มี address space ของตัวเอง
c) ป้องกันไม่ให้ process หนึ่งเข้าถึง memory ของอีก process
d) เพิ่ม hit rate ของ L1 cache

V2. Page fault ถูกจัดการโดยใคร

a) Cache controller
b) TLB
c) Operating system (software)
d) Compiler

V3. ทำไม virtual memory จึงใช้ write-back เท่านั้น

a) write-through ไปยังดิสก์ช้าและแพงเกินไป
b) ดิสก์เขียนข้อมูลไม่ได้
c) OS ติดตาม dirty page ไม่ได้
d) TLB บังคับไว้

V4. TLB คืออะไร

a) cache ของข้อมูล
b) cache ของ page table entries ที่เพิ่งใช้
c) ส่วนหนึ่งของ disk
d) register เก็บ PC

V5. TLB มักจัดเป็น fully associative เพราะ

a) ขนาดใหญ่มาก

b) มีขนาดเล็ก (ประมาณ 128–256 entries) จึงเทียบทุก entry ได้

c) ไม่มี tag

d) OS จัดการทั้งหมด

V6. ข้อใดถูกเกี่ยวกับ page table

a) มีตารางเดียวใช้ร่วมทั้งระบบ

b) มีหนึ่งตารางต่อ process และมี page table register ชี้ไปที่ตารางปัจจุบัน

c) อยู่ใน TLB ทั้งหมด

d) เก็บข้อมูลของโปรแกรม

V7. ปัญหาของ cache ที่ใช้ virtual address (ก่อนแปล) คือ

a) ช้ากว่า physically addressed เสมอ

b) aliasing: VA ต่างกันจากต่าง process อาจ map ไป PA เดียวกัน (shared memory)

c) ไม่มี tag

d) ใช้กับ write-back ไม่ได้

ข. คำนวณ

V8. VA 32 bits, page 8 KB, PTE 4 bytes ขนาด page table ต่อ process คือ

a) 1 MB

b) 2 MB

c) 4 MB

d) 8 MB

V9. page 4 KB และต้องการ L1 cache 16 KB ที่ทำ overlapped access ได้ ต้องใช้ associativity อย่างน้อยเท่าใด

a) 2-way

b) 4-way

c) 8-way

d) 16-way

ค. ตอบสั้น

V10. VA 32 bits, page 4 KB, physical memory 1 GB, PTE 4 bytes จงหาจำนวนบิตของ VPN, offset, PPN และขนาด page table

---

V11. VA 32 bits, page 16 KB, PTE 4 bytes จงหาจำนวนบิตของ VPN และขนาด page table

---

V12. (Exercise ในสไลด์) page 4 KB สถาปนิกควรออกแบบ cache 32 KB อย่างไรให้ทำ overlapped access ได้

V13. (สไลด์) cache 8 KB direct-mapped กับ page 4 KB ทำ overlapped access ไม่ได้เพราะอะไร และมีทางแก้อะไรบ้าง

V14. (Exercise ในสไลด์) เมื่อ OS ทำ process switch ต้องทำอะไรกับ TLB, page table และ cache

---

เฉลย: Ch8 (2): Virtual Memory และ TLB

ข้อ	V1	V2	V3	V4	V5	V6	V7	V8	V9
ตอบ	d	c	a	b	b	b	b	b	b

V1. ตอบ d) เพิ่ม hit rate ของ L1 cache

a, b, c คือ cost, relocation และ protection

V2. ตอบ c) Operating system (software)

hardware ไม่รู้จัก file system หรือ swap

V3. ตอบ a) write-through ไปยังดิสก์ช้าและแพงเกินไป

ถ้า write-through ทุกการเขียนต้องไปถึงดิสก์

V4. ตอบ b) cache ของ page table entries ที่เพิ่งใช้

ทำให้การแปล VA → PA เร็วพอ ๆ กับการเข้า cache

V5. ตอบ b) มีขนาดเล็ก (ประมาณ 128–256 entries) จึงเทียบทุก entry ได้

เครื่องระดับกลางอาจใช้ small n-way

V6. ตอบ b) มีหนึ่งตารางต่อ process และมี page table register ชี้ไปที่ตารางปัจจุบัน

ตอน process switch จึงแค่เปลี่ยน register นี้

V7. ตอบ b) aliasing: VA ต่างกันจากต่าง process อาจ map ไป PA เดียวกัน (shared memory)

และต้องจัดการตอน context switch ด้วย

V8. ตอบ b) 2 MB

offset 13 bits → VPN 19 bits → $2^{19} \times 4$ B = 2 MB (4 MB คือกรณี page 4 KB)

V9. ตอบ b) 4-way

16 KB / ways ≤ 4 KB

V10. **VPN 20, offset 12, PPN 18, page table 4 MB**

PA = 30 bits → PPN = 30 − 12 (ตรงกับตัวอย่างในสไลด์)

V11. **VPN 18 bits, page table 1 MB**

offset 14 bits · page ใหญ่ขึ้น page table เล็กลง

V12. **8-way set associative (4 KB ต่อ way)**

index + offset ต้องอยู่ใน page offset 12 bits · ทางเลือกอื่นคือใช้ page ใหญ่ขึ้น หรือให้ OS รับประกันบิต VA = PA (page coloring)

V13. **cache ใช้ 13 bits แต่ page offset มี 12 bits · แก้: page 8 KB, 2-way set associative, หรือ SW รับประกัน VA[12] = PA[12]**

บิตที่ 12 ของ VA ถูกเปลี่ยนตอนแปลงเป็น PA จึงใช้เป็น index ก่อนแปลไม่ได้

V14. **TLB: flush (หรือใช้ ASID) · page table: ไม่ลบ แค่เปลี่ยน page table register · cache: physical ไม่ต้อง flush ส่วน virtual ต้อง flush หรือใส่ PID**

mapping ใน TLB เป็นของ process เก่า

Ch9: Storage (RAID, SAN, NAS)

สรุปก่อนทำโจทย์

- **RAID** = Redundant Array of Independent (เดิม Inexpensive) Disks (Patterson, Gibson, Katz, UC Berkeley, 1987) เป็น storage virtualization ที่ทำให้ดิสก์ราคาถูกได้ performance เทียบดิสก์แพง

Level	วิธี	Min	ทนเสีย	Capacity
0	Stripe	2	0	N
1	Mirror	2	1	N/2
2	Bit-level + Hamming	—	1	(แทบไม่ใช้)
3	Byte-level parity	3	1	N – 1
4	Block parity บนดิสก์เฉพาะ	3	1	N – 1 (write ช้า: parity disk เป็นคอขวด)
5	Block parity **กระจายทุกดิสก์**	**3**	1	**N – 1**
6	2 parities	**4**	**2**	**N – 2**

- **Nested:** RAID 01, 10, 50 · **JBOD/span:** รวมดิสก์ต่างขนาดได้แต่ไม่มี redundancy · **RAID ไม่ใช่ backup**
- **SAN** = **block** storage (iSCSI, Fibre Channel) client จัดการ file system มักไม่ share · **NAS** = **file** storage (NFS, CIFS/SMB) server จัดการ file system share ง่าย
- **Cloud:** EBS = block · S3 = object · Google Drive/Dropbox = file · Protocol/interface (SATA, SAS, PCIe) ≠ storage technology (SSD, HDD)

โจทย์ (13 ข้อ)

ก. RAID

R1. RAID 0 มีลักษณะอย่างไร

a) Mirror ทนเสียได้ 1 ลูก
b) Stripe เร็วที่สุดแต่ไม่มี redundancy
c) Parity กระจายทุกดิสก์
d) Parity สองชุด

R2. ทำไม RAID 4 จึงเขียนช้า

a) ไม่มี parity
b) parity อยู่บนดิสก์เฉพาะลูกเดียว ทุกการเขียนต้องใช้ดิสก์นั้นจึงเป็นคอขวด
c) ใช้ bit-level parity
d) ต้องใช้ดิสก์ 2 ลูก

R3. RAID 5 ที่ใช้ดิสก์ 2 TB จำนวน 4 ลูก มีพื้นที่ใช้งานจริงเท่าใด

a) 2 TB
b) 4 TB
c) 6 TB
d) 8 TB

R4. RAID 6 ที่ใช้ดิสก์ 4 TB จำนวน 6 ลูก มีพื้นที่ใช้งานจริงเท่าใด

a) 24 TB
b) 20 TB
c) 16 TB
d) 12 TB

R5. ข้อใดถูกเกี่ยวกับ JBOD

a) ทนดิสก์เสียได้ 1 ลูก
b) รวมดิสก์ต่างขนาดเป็นก้อนเดียวได้ แต่ไม่มี redundancy
c) เป็น RAID level 7
d) ใช้ parity สองชุด

ข. SAN, NAS และ Cloud

R6. NAS ใช้ protocol ใดในการแชร์ไฟล์

a) iSCSI
b) Fibre Channel
c) NFS
d) SATA

R7. ใน SAN ใครเป็นผู้จัดการ file system

a) Server ของ storage
b) Client (OS ของเครื่องที่ใช้ เห็นเป็น block)
c) Switch ของเครือข่าย
d) ไม่มี file system

R8. Amazon EBS และ NAS ที่ใช้ NFS ให้บริการ storage ระดับใดตามลำดับ

a) Block, File
b) File, Block
c) Object, File
d) Block, Object

R9. Amazon S3 เป็น storage ประเภทใด

a) Block
b) File
c) Object
d) RAID

ค. ตอบสั้น

R10. ทำไม RAID 5 จึงเขียนได้เร็วกว่า RAID 4 และ RAID 5 ต้องใช้ดิสก์อย่างน้อยกี่ลูก

R11. ดิสก์ 3 TB จำนวน 5 ลูก จงหาพื้นที่ใช้งานและจำนวนดิสก์ที่เสียได้ สำหรับ RAID 0, 5 และ 6

R12. (Exercise ในสไลด์) RAID 0, 1 และ 5 ต้องใช้ดิสก์อย่างน้อยกี่ลูก

R13. เปรียบเทียบ SAN กับ NAS ใน 3 ประเด็น: ระดับของ storage, ใครจัดการ file system, การ share

เฉลย: Ch9: Storage (RAID, SAN, NAS)

ข้อ	R1	R2	R3	R4	R5	R6	R7	R8	R9
ตอบ	b	b	c	c	b	c	b	a	c

R1. ตอบ b) Stripe เร็วที่สุดแต่ไม่มี redundancy

เสียลูกเดียวข้อมูลหายหมด

R2. ตอบ b) parity อยู่บนดิสก์เฉพาะลูกเดียว ทุกการเขียนต้องใช้ดิสก์นั้นจึงเป็นคอขวด

RAID 5 แก้ด้วยการกระจาย parity ไปทุกดิสก์

R3. ตอบ c) 6 TB

(4 – 1) × 2 (8 TB = RAID 0 และ 4 TB = RAID 1/10)

R4. ตอบ c) 16 TB

(6 – 2) × 4 และทนดิสก์เสียได้ 2 ลูก

R5. ตอบ b) รวมดิสก์ต่างขนาดเป็นก้อนเดียวได้ แต่ไม่มี redundancy

JBOD = Just a Bunch Of Disks (span)

R6. ตอบ c) NFS

iSCSI เป็นของ SAN

R7. ตอบ b) Client (OS ของเครื่องที่ใช้ เห็นเป็น block)

NAS เป็นแบบตรงข้าม คือ server จัดการ file system และ permission

R8. ตอบ a) Block, File

EBS เหมือนดิสก์เปล่าที่ต่อกับ VM

R9. ตอบ c) Object

เข้าถึงผ่าน API เป็น object (ข้อมูล + metadata + ID)

R10. **parity กระจายทุกดิสก์จึงเขียนหลาย stripe ขนานกันได้ (RAID 4 มี parity disk เป็นคอขวด) · อย่างน้อย 3 ลูก**

ทั้งคู่ทนดิสก์เสียได้ 1 ลูก

R11. **RAID 0: 15 TB เสียได้ 0 · RAID 5: 12 TB เสียได้ 1 · RAID 6: 9 TB เสียได้ 2**

N, N – 1 และ N – 2 ลูก

R12. **2, 2, 3**

RAID 6 ต้องใช้อย่างน้อย 4

R13. **SAN: block, client จัดการ FS, ไม่นิยม share · NAS: file, server จัดการ FS, share ง่าย**

SAN ใช้ iSCSI/Fibre Channel ส่วน NAS ใช้ NFS/SMB